Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 16 (Final Volume), Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 advanced troubleshooting scenarios, 10 enterprise production incident cases, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your own platform project. That is where the learning happens.

মূল্যায়ন নির্দেশিকা • ভলিউম ১৬

Assessment & Answer Guide — পরিচিতি ও ব্যবহারের নিয়ম

১.১ এই নির্দেশিকা কীভাবে ব্যবহার করবেন

এই অংশটি ভলিউম ১৬-এর ২৪টি অধ্যায়ের মূল্যায়ন ও দ্রুত স্মরণের হাতিয়ার। এটি শুধু মুখস্থ যাচাই করে না — এটি যাচাই করে আপনি একটি স্থাপত্য সিদ্ধান্ত নিতে, তার পক্ষে যুক্তি দিতে, ভুল হলে তার পরিণতি ব্যাখ্যা করতে এবং প্রোডাকশনে সেই সিদ্ধান্ত রক্ষা করতে পারেন কি না। প্রতিটি প্রশ্নের পাশে কঠিনতা (Difficulty) দেওয়া আছে: **Beginner** — ধারণা বোঝা; **Intermediate** — প্রয়োগ ও তুলনা; **Advanced** — ট্রেড-অফ, ব্যর্থতা ও সিদ্ধান্ত রক্ষা।

পড়ার সুপারিশ: প্রথমে অধ্যায়গুলো পড়ুন, তারপর MCQ দিয়ে নিজেকে যাচাই করুন, তারপর Short Answer লিখে দেখুন — লিখতে না পারলে ধারণাটি এখনও আপনার হয়নি। Scenario ও Case Study-গুলো দলে আলোচনা করুন; একা পড়ার চেয়ে আলোচনায় সিদ্ধান্তের দুর্বলতা দ্রুত ধরা পড়ে।

> **CONCEPT:** ভালো স্থপতি সেই নয় যে সবচেয়ে বেশি টুলের নাম জানে; ভালো স্থপতি সেই যে একটি সিদ্ধান্তের খরচ, ঝুঁকি ও বিকল্প বলতে পারে। এই গাইডের Advanced প্রশ্নগুলো ঠিক সেই দক্ষতাই মাপে।

১.২ পরীক্ষার নিয়ম

- MCQ: প্রতিটি প্রশ্নে একটি সঠিক উত্তর। অনুমান করে উত্তর দেওয়ার চেয়ে “জানি না” স্বীকার করা শেখার জন্য ভালো — তবে পরীক্ষায় প্রতিটি প্রশ্নের উত্তর দিন।
- Short Answer: প্রতিটি উত্তর ৩–৫ বাক্যে, উদাহরণসহ লিখুন। শুধু সংজ্ঞা দিলে অর্ধেক নম্বর।
- Scenario ও Case Study: উত্তর দেওয়ার আগে পরিস্থিতির তথ্য, অনুমান ও ঝুঁকি আলাদা করে লিখুন। “কী করব” বলার আগে “কেন” বলুন।
- Final Exam: ১০০ নম্বর, ৩ ঘণ্টা। কোনো বই বা ইন্টারনেট ছাড়া, শুধু এই ভলিউমের হ্যান্ডবুক অংশটি সাথে রাখা যাবে।
- সততা: অন্যের উত্তর দেখে লেখা নিজের দুর্বলতা লুকায় মাত্র। মূল্যায়নের উদ্দেশ্য দুর্বলতা খুঁজে বের করা, লুকানো নয়।

১.৩ নম্বর বিভাজন ও পাসের মানদণ্ড

অংশ	নম্বর	সময় (পরামর্শ)
MCQ (১২০টি)	১২০	৯০ মিনিট
Short Answer (৩০টি)	৬০	৬০ মিনিট
Scenario (২০টি)	৬০	৯০ মিনিট
Case Study (১০টি)	৪০	৬০ মিনিট
Final Exam (আলাদা)	১০০	৩ ঘণ্টা

স্ব-মূল্যায়নে ৭০% বা তার বেশি পেলে আপনি পরের ভলিউম বা বাস্তব প্রকল্পের জন্য প্রস্তুত বলে ধরা যায়। ৫০%-এর নিচে পেলে সংশ্লিষ্ট অধ্যায়গুলো আবার পড়ুন — Answer Guide-2-এর chapter mapping দেখে নিন কোন অধ্যায়ে ফিরতে হবে।

> **WARNING:** উত্তরপত্রে (Answer Guide) শুধু সঠিক অক্ষর দেখে মুখস্থ করবেন না। প্রতিটি ব্যাখ্যা পড়ুন — একই প্রশ্ন প্রোডাকশনে অন্যভাবে এলে ব্যাখ্যাটিই আপনাকে সঠিক সিদ্ধান্ত নিতে সাহায্য করবে।

মূল্যায়ন • ১২০টি প্রশ্ন

Multiple Choice Questions — ১২০টি প্রশ্ন (প্রতি অধ্যায়ে ৫টি)

প্রতিটি প্রশ্নে চারটি বিকল্প (A, B, C, D)। সঠিক উত্তরগুলো A, B, C, D-তে সমানভাবে বণ্টিত — প্রতিটিতে ৩০টি করে। কঠিনতা প্রতিটি প্রশ্নের পাশে দেওয়া আছে। উত্তরপত্র Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. TechNova-র প্রোডাকশন ক্লাস্টারে API server ধীর হয়ে গেলে প্রথমে কোন উপাদানটি পরীক্ষা করা যুক্তিসংগত? (*Intermediate* • অধ্যায় 1)

1. etcd-র লেটেন্সি ও ডিস্ক I/O, কারণ API server-এর প্রতিটি লেখা etcd-তে যায়
2. kube-proxy-র iptables নিয়ম, কারণ সেটিই API অনুরোধ রুট করে
3. kubelet-র লগ, কারণ সেটি API server চালায়
4. CoreDNS-এর ক্যাশ, কারণ সেটি API অনুরোধ ক্যাশ করে

2. একটি ক্লাস্টারে control plane-কে highly available করতে সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত কোনটি? (*Advanced* • অধ্যায় 1)

1. kubelet-র সংখ্যা বাড়ানো, যাতে বেশি Pod চলতে পারে
2. etcd-কে অন্তত তিনটি সদস্যের quorum-সহ আলাদা নোডে চালানো এবং API server-কে লোড ব্যালান্সারের পেছনে রাখা
3. সব control plane উপাদান একই নোডে রাখা, যাতে যোগাযোগ দ্রুত হয়
4. etcd-কে একটি মাত্র নোডে রেখে ব্যাকআপ নেওয়া, যাতে quorum জটিলতা না থাকে

3. kube-scheduler Pod-কে কোন নোডে বসাবে তা ঠিক করার সময় কী বিবেচনা করে না? (*Beginner* • অধ্যায় 1)

1. নোডের উপলব্ধ CPU ও মেমোরি এবং resource request
2. affinity, taint ও toleration নিয়ম
3. Pod-এর ভেতরে চলা অ্যাপ্লিকেশনের সোর্স কোডের গুণমান
4. নোডের লেবেল ও Pod-এর nodeSelector

4. API server-এ admission webhook টাইমআউট হলে নিরাপদ ডিফল্ট আচরণ কোনটি? (*Advanced* • অধ্যায় 1)

1. সব webhook উপেক্ষা করে অনুরোধ গ্রহণ করা, যাতে ডেপ্লয় না থামে
2. Webhook-কে বারবার চেষ্টা করা যতক্ষণ না ক্লাস্টার অচল হয়
3. Admission নিয়ন্ত্রণ পুরোপুরি বন্ধ করে দেওয়া
4. Validating webhook ব্যর্থ হলে অনুরোধ প্রত্যাখ্যান করা (fail-closed), যদি নীতিটি নিরাপত্তা-সংবেদনশীল হয়

5. TechNova-র ক্লাস্টারে হঠাৎ সব Pod Pending অবস্থায় আটকে গেলে প্রথম যাচাই কোনটি? (*Intermediate* • অধ্যায় 1)

1. Pod-এর image tag, কারণ image ভুল হলে Pod Pending হয়
2. Scheduler-র লগ ও নোডের উপলব্ধ resource — resource request মেটানোর মতো নোড আছে কি না

3. Ingress-র TLS সার্টিফিকেট, কারণ সেটি ছাড়া Pod শুরু হয় না
4. Git রিপোজিটরির শাখার নাম

6. etcd-র ডেটা নষ্ট হলে TechNova-র পুনরুদ্ধারের সবচেয়ে নির্ভরযোগ্য পথ কোনটি? (*Intermediate* • অধ্যায় 2)

1. চলমান Pod-গুলো থেকে ক্লাস্টারের অবস্থা পুনর্গঠন করা
2. kubelet-র ক্যাশ থেকে YAML আবার তৈরি করা
3. নিয়মিত নেওয়া etcd snapshot থেকে পুনরুদ্ধার করা এবং snapshot-এর বয়স যাচাই করা
4. API server পুনরায় চালু করলেই ডেটা ফিরে আসবে বলে ধরে নেওয়া

7. etcd-র জন্য সবচেয়ে উপযুক্ত ডিস্ক কোনটি? (*Beginner* • অধ্যায় 2)

1. বড় ধারণক্ষমতার HDD, কারণ etcd-তে অনেক ডেটা জমে
2. নেটওয়ার্ক শেয়ার্ড ডিস্ক, কারণ সব নোড তা দেখতে পায়
3. RAM disk, কারণ সেটি সবচেয়ে দ্রুত
4. কম লেটেন্সির SSD, কারণ etcd প্রতিটি লেখায় fsync করে

8. API server-এ RBAC অনুমোদন ব্যর্থ হলে একজন ডেভেলপার কী দেখবেন? (*Beginner* • অধ্যায় 2)

1. 403 Forbidden — প্রমাণীকরণ সফল কিন্তু অনুমতি নেই
2. 401 Unauthorized — সবসময় টোকেনের মেয়াদ শেষ হয়েছে
3. 404 Not Found — রিসোর্সটি নেই
4. 500 Internal Error — API server ভেঙে গেছে

9. etcd quorum হারালে কী ঘটে? (*Intermediate* • অধ্যায় 2)

1. API server স্বয়ংক্রিয়ভাবে নতুন etcd তৈরি করে নেয়
2. Pod-গুলো চলা বন্ধ করে দেয়, কিন্তু ডেটা নিরাপদ থাকে
3. etcd শুধু পড়ার (read-only) অবস্থায় যায় বা লেখা বন্ধ করে দেয়, যাতে বিভক্ত মস্তিষ্ক (split-brain) না হয়
4. Scheduler নতুন Pod বানাতে থাকে, পুরোনো অবস্থা উপেক্ষা করে

10. API aggregation layer ব্যবহারের প্রধান সুবিধা কোনটি? (*Advanced* • অধ্যায় 2)

1. API server-র গতি দশগুণ বাড়ে
2. etcd-র প্রয়োজন দূর হয়
3. RBAC আর প্রয়োজন হয় না
4. তৃতীয় পক্ষের API সার্ভারকে মূল API-তে যুক্ত করে নতুন রিসোর্স ধরন দেওয়া যায়, মূল API server বদল না করে

11. TechNova ঢাকা ও চট্টগ্রামে দুটি ক্লাস্টার চালায়। একটি ক্লাস্টার পুরো অচল হলে ট্রাফিক সরানোর সবচেয়ে নিরাপদ কৌশল কোনটি? (*Advanced* • অধ্যায় 3)

1. আগে থেকে পরীক্ষা করা DNS/ Global Load Balancer failover, যাতে স্বাস্থ্য-পরীক্ষা ব্যর্থ হলে ট্রাফিক অন্য ক্লাস্টারে যায়
2. অচল ক্লাস্টারের kubeconfig ম্যানুয়ালি সম্পাদনা করা
3. দুটি ক্লাস্টারের etcd একত্র করে দেওয়া

4. অপেক্ষা করা — Kubernetes নিজেই অন্য ক্লাস্টারে Pod বানিয়ে দেবে

12. Multi-cluster পরিচালনায় প্রতিটি ক্লাস্টারে একই নীতি প্রয়োগের সবচেয়ে টেকসই উপায় কোনটি? (*Intermediate* • অধ্যায় 3)

1. প্রতিটি ক্লাস্টারে হাতে kubectl apply করা
2. একটি কেন্দ্রীয় Git রিপোজিটরি থেকে প্রতিটি ক্লাস্টারে GitOps দিয়ে একই নীতি ও কনফিগারেশন প্রয়োগ করা
3. একজন প্রকৌশলী সব ক্লাস্টারে একই কমান্ড মুখস্থ চালান
4. নীতি শুধু একটি ক্লাস্টারে রেখে বাকিগুলোতে বিশ্বাস করা

13. Cluster federation-এর বদলে প্রতিটি ক্লাস্টার স্বাধীন রাখার যুক্তি কোনটি সবচেয়ে শক্তিশালী? (*Intermediate* • অধ্যায় 3)

1. স্বাধীন ক্লাস্টারে খরচ সবসময় কম হয়
2. স্বাধীন ক্লাস্টারে নেটওয়ার্কের প্রয়োজন হয় না
3. Federation কখনোই কাজ করে না
4. একটি ক্লাস্টারের ব্যর্থতার বিস্ফোরণ-ব্যাসার্ধ (blast radius) অন্যটিতে ছড়ায় না

14. দুটি ক্লাস্টারের মধ্যে সার্ভিস-টু-সার্ভিস যোগাযোগের জন্য সবচেয়ে কম জটিল প্রাথমিক পদ্ধতি কোনটি? (*Beginner* • অধ্যায় 3)

1. Ingress/Gateway দিয়ে প্রকাশিত HTTPS API দিয়ে যোগাযোগ, প্রয়োজনে mTLS যুক্ত করে
2. দুটি ক্লাস্টারের Pod নেটওয়ার্ক সরাসরি এক করে দেওয়া
3. etcd দুটি ক্লাস্টারে শেয়ার করা
4. একই Service নাম দুই জায়গায় রাখলেই যোগাযোগ হয় বলে ধরে নেওয়া

15. Multi-cluster পরিবেশে কোন বিষয়টি সবচেয়ে বেশি পরিচালন-ভার বাড়ায়? (*Intermediate* • অধ্যায় 3)

1. Pod-এর সংখ্যা গোনা
2. প্রতিটি ক্লাস্টারের সংস্করণ, নীতি, পর্যবেক্ষণ ও আপগ্রেডের ভিন্নতা সামলানো
3. একটি ক্লাস্টারে Namespace-এর নাম দেওয়া
4. kubectl-র সংস্করণ মনে রাখা

16. GitOps-এ Git রিপোজিটরি কোন ভূমিকা পালন করে? (*Beginner* • অধ্যায় 4)

1. শুধু অ্যাপ কোড জমা রাখার জায়গা
2. লগ সংরক্ষণের জায়গা
3. সিস্টেমের কাঙ্ক্ষিত অবস্থার একমাত্র সত্যের উৎস (single source of truth)
4. সিক্রেট সংরক্ষণের নিরাপদ জায়গা — প্লেইন টেক্সটে

17. Argo CD-তে একটি অ্যাপ্লিকেশন OutOfSync দেখাচ্ছে, কিন্তু কেউ ক্লাস্টারে কিছু বদলায়নি। সবচেয়ে সম্ভাব্য কারণ কোনটি? (*Beginner* • অধ্যায় 4)

1. Git-এ কেউ নতুন কমিট করেছে, অথবা Argo CD-র sync সেটিংসে automated sync বন্ধ আছে
2. Argo CD সার্ভার বন্ধ হয়ে গেছে

3. Pod-গুলো CrashLoopBackOff-এ আছে
4. Ingress-র DNS রেকর্ড মুছে গেছে

18. Argo CD-তে self-heal চালু থাকলে কী হয়? (*Intermediate* • অধ্যায় 4)

1. Git-এ করা পরিবর্তন স্বয়ংক্রিয়ভাবে বাতিল হয়
2. Pod মুছে গেলে আর তৈরি হয় না
3. ক্লাস্টারে হাতে করা পরিবর্তন Argo CD স্বয়ংক্রিয়ভাবে Git-এর অবস্থায় ফিরিয়ে আনে
4. Sync শুধু ম্যানুয়ালি করা যায়

19. TechNova-র প্রোডাকশনে GitOps-এর সবচেয়ে বড় ঝুঁকি কোনটি, আর তার প্রতিকার কী? (*Advanced* • অধ্যায় 4)

1. GitOps-এ কোনো ঝুঁকি নেই, কারণ Git সবসময় নিরাপদ
2. ঝুঁকি শুধু নেটওয়ার্কে; Git ভুল হতে পারে না
3. প্রতিকার হলো GitOps বন্ধ করে হাতে kubectl চালানো
4. Git রিপোজিটরিতে ভুল কনফিগারেশন গেলে তা স্বয়ংক্রিয়ভাবে প্রোডাকশনে ছড়িয়ে পড়ে — তাই Pull Request review, automated sync-এ প্রুন ও self-heal সতর্কভাবে, এবং প্রয়োজনে manual approval ধাপ রাখা

20. Argo CD ApplicationSet কখন ব্যবহার করা যুক্তিসংগত? (*Intermediate* • অধ্যায় 4)

1. একটি মাত্র অ্যাপ, একটি ক্লাস্টারে চালাতে হলে
2. একই অ্যাপ অনেক ক্লাস্টার বা পরিবেশে টেমপ্লেট থেকে তৈরি করতে হলে
3. শুধু Helm chart ইনস্টল করতে হলে
4. Git রিপোজিটরি ছাড়া কাজ করতে হলে

21. Flux-এ GitRepository ও Kustomization রিসোর্সের সম্পর্ক কী? (*Beginner* • অধ্যায় 5)

1. Kustomization সোর্স নির্ধারণ করে, GitRepository প্রয়োগ করে
2. GitRepository সোর্স নির্ধারণ করে, Kustomization সেই সোর্স থেকে কী প্রয়োগ করতে হবে তা নির্ধারণ করে
3. দুটো একই কাজ করে, একটি ব্যাকআপ হিসেবে থাকে
4. GitRepository শুধু Helm chart-এর জন্য, Kustomization শুধু YAML-এর জন্য

22. Flux-এর reconciliation interval খুব ছোট রাখলে কী সমস্যা হয়? (*Intermediate* • অধ্যায় 5)

1. Sync আরও নিরাপদ হয়, কোনো সমস্যা হয় না
2. Flux আর কাজ করে না
3. Pod-গুলো পুনরায় চালু হয়
4. Git ও API server-এ অপ্রয়োজনীয় চাপ বাড়ে এবং ছোট ভুল দ্রুত ছড়িয়ে পড়ে

23. Flux Image Automation ব্যবহারের প্রধান সতর্কতা কোনটি? (*Advanced* • অধ্যায় 5)

1. নতুন image স্বয়ংক্রিয়ভাবে প্রোডাকশনে গেলে ভাঙা রিলিজ ঢুকে পড়তে পারে — তাই image policy ও পরীক্ষার ধাপ জরুরি
2. Image Automation ব্যবহার করলে Git-এর প্রয়োজন হয় না

3. এটি শুধু public image-এ কাজ করে
4. এটি কখনোই ভুল image নেয় না

24. Argo CD-র বদলে Flux বেছে নেওয়ার একটি যুক্তিসংগত কারণ কোনটি? (*Intermediate* • অধ্যায় 5)

1. Flux-এ কখনো sync ব্যর্থ হয় না
2. Argo CD-তে Git ব্যবহার করা যায় না
3. দলটি GitOps Toolkit-এর কম্পোজেবল, CLI-কেন্দ্রিক পদ্ধতি চায় এবং UI-র চেয়ে Kubernetes নেটিভ রিসোর্সকে অগ্রাধিকার দেয়
4. Flux-এ RBAC-এর প্রয়োজন হয় না

25. Flux-এ একটি Kustomization ক্রমাগত reconciliation ব্যর্থ হলে প্রথমে কী দেখবেন? (*Beginner* • অধ্যায় 5)

1. Flux controller-র লগ ও Kustomization-এর status/conditions — কোন সোর্স বা YAML-এ ত্রুটি আছে
2. Pod-এর image tag
3. Ingress-র TLS সার্টিফিকেট
4. নোডের ডিস্কের আকার

26. Helm chart-এর values.yaml-এর মূল উদ্দেশ্য কী? (*Beginner* • অধ্যায় 6)

1. Chart-এর সোর্স কোড লুকানো
2. একই chart-কে পরিবেশভেদে ভিন্ন মান দিয়ে পুনর্ব্যবহার করা
3. Kubernetes-এর সংস্করণ নির্ধারণ করা
4. Pod-এর লগ সংরক্ষণ করা

27. Kustomize-এর সবচেয়ে বড় শক্তি কোনটি? (*Beginner* • অধ্যায় 6)

1. এটি টেমপ্লেট ভাষা শেখায়
2. এটি Helm chart ইনস্টল করে
3. মূল YAML না বদলে overlay দিয়ে পরিবেশভেদে প্যাচ, নাম ও রেপ্লিকা বদলানো যায়
4. এটি সিক্রেট এনক্রিপ্ট করে

28. TechNova একটি তৃতীয় পক্ষের জটিল অ্যাপ (অনেক টেমপ্লেট চলকসহ) চালাতে চায়। Helm না Kustomize — কোনটি যুক্তিসংগত এবং কেন? (*Intermediate* • অধ্যায় 6)

1. Kustomize, কারণ তৃতীয় পক্ষের chart কখনোই Helm-এ চলে না
2. কোনোটিই নয় — হাতে YAML লেখাই একমাত্র উপায়
3. Helm শুধু নিজের অ্যাপের জন্য
4. Helm, কারণ তৃতীয় পক্ষের প্যাকেজ করা অ্যাপে values দিয়ে কনফিগার করা ও সংস্করণ ধরে রাখা সহজ

29. Helm release আপগ্রেড ব্যর্থ হয়ে আটকে গেলে নিরাপদ প্রথম পদক্ষেপ কোনটি? (*Intermediate* • অধ্যায় 6)

1. Chart মুছে আবার ইনস্টল করা, ডেটা হারানোর ঝুঁকি নিয়ে
2. helm rollback দিয়ে আগের পরিচিত ভালো revision-এ ফেরা এবং ব্যর্থতার কারণ লগে যাচাই করা
3. Release-এর Secret হাতে সম্পাদনা করা

4. Kubernetes ক্লাস্টার পুনরায় চালু করা

30. Helm chart-এ সিক্রেট প্লেইন টেক্সট values-এ লেখা কেন বিপজ্জনক? (*Intermediate* • অধ্যায় 6)

1. Helm সিক্রেট সমর্থন করে না
2. Values ফাইল কখনোই পড়া যায় না
3. Values ফাইল Git-এ যায় ও helm history-তে সংরক্ষিত হয় — সিক্রেট ফাঁস হয়ে যায়
4. সিক্রেট লিখলে chart ইনস্টল হয় না

31. Service mesh-এর data plane ও control plane-র পার্থক্য কী? (*Beginner* • অধ্যায় 7)

1. Data plane নীতি লেখে, control plane ট্রাফিক বহন করে
2. দুটো একই জিনিস, নাম ভিন্ন
3. Control plane Pod-এর ভেতরে চলে, data plane নোডে চলে
4. Data plane (sidecar proxy) ট্রাফিক বহন করে; control plane (যেমন Istiod) proxy-দের কনফিগারেশন দেয়

32. mTLS চালু করার পর দুটি সার্ভিসের যোগাযোগ ব্যর্থ হলে প্রথমে কী যাচাই করবেন? (*Intermediate* • অধ্যায় 7)

1. উভয় Pod-এ sidecar proxy আছে কি না এবং destination rule-এ mTLS মোড কী
2. সার্ভিসের সোর্স কোডে TLS লাইব্রেরি আছে কি না
3. Ingress-র DNS রেকর্ড
4. ডেটাবেসের পাসওয়ার্ড

33. TechNova-র ছোট দল, মাত্র তিনটি সার্ভিস, সীমিত অপারেশন ক্ষমতা — service mesh নেওয়া কি যুক্তিসংগত? (*Advanced* • অধ্যায় 7)

1. অবশ্যই হ্যাঁ — mesh ছাড়া মাইক্রোসার্ভিস চলে না
2. হ্যাঁ, কারণ mesh সব সমস্যা সমাধান করে
3. সম্ভবত নয় — mesh-এর পরিচালন-ভার সুবিধার চেয়ে বেশি হতে পারে; প্রথমে Ingress + mTLS-এর সহজ বিকল্প বিবেচনা করুন
4. না, কারণ ছোট দল কখনোই TLS ব্যবহার করতে পারে না

34. Istio-তে traffic shifting (canary) করার সময় কোন রিসোর্সটি মূলত ব্যবহৃত হয়? (*Beginner* • অধ্যায় 7)

1. PodDisruptionBudget
2. NetworkPolicy
3. StorageClass
4. VirtualService — ওজন (weight) দিয়ে ট্রাফিক ভাগ করা হয়

35. Service mesh-এ observability-র সবচেয়ে বড় সুবিধা কোনটি? (*Intermediate* • অধ্যায় 7)

1. অ্যাপ কোড না বদলে প্রতিটি সার্ভিসের লেটেন্সি, ত্রুটি ও ট্রাফিকের স্বয়ংক্রিয় telemetry পাওয়া যায়
2. Mesh অ্যাপের বাগ নিজেই ঠিক করে দেয়
3. Mesh ডেটাবেসের গতি বাড়িয়ে দেয়

4. Mesh কোড লেখা কমিয়ে দেয়

36. Ingress-এর বদলে Gateway API বেছে নেওয়ার সবচেয়ে শক্তিশালী যুক্তি কোনটি? (*Advanced* • অধ্যায় 8)

1. Gateway API সবসময় দ্রুততর
2. Gateway API ভূমিকা-ভিত্তিক (infrastructure, cluster operator, developer) ও এক্সটেনসিবল — Ingress-এর annotation-নির্ভর সীমা কাটায়
3. Ingress এখন আর কাজ করে না
4. Gateway API-তে TLS-এর প্রয়োজন হয় না

37. CNI প্লাগইন বেছে নেওয়ার সময় সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন কোনটি? (*Intermediate* • অধ্যায় 8)

1. CNI-র নামটি কত সুন্দর
2. CNI কতগুলো দেশে ব্যবহৃত হয়
3. CNI-র লোগোর রং কী
4. ক্লাস্টারের নেটওয়ার্ক নীতি (NetworkPolicy), এনক্রিপশন ও পারফরম্যান্সের প্রয়োজন কী

38. NetworkPolicy প্রয়োগের পর একটি সার্ভিস অন্য সার্ভিসকে ডাকতে পারছে না। প্রথম যাচাই কোনটি? (*Intermediate* • অধ্যায় 8)

1. Policy-র podSelector ও ingress/egress নিয়ম — অনুমোদিত উৎস ও পোর্ট ঠিক আছে কি না
2. সার্ভিসের সোর্স কোড
3. DNS সার্ভারের অবস্থান
4. নোডের CPU ব্যবহার

39. TechNova-র API-কে বাইরে প্রকাশ করতে সবচেয়ে নিরাপদ প্রাথমিক ব্যবস্থা কোনটি? (*Beginner* • অধ্যায় 8)

1. প্রতিটি Pod-কে সরাসরি ইন্টারনেটে প্রকাশ করা
2. TLS সমাপ্তি (termination) ও প্রমাণীকরণ Ingress/Gateway স্তরে কেন্দ্রীয়ভাবে করা
3. TLS ছাড়া HTTP-এ চালানো, কারণ ভেতরের নেটওয়ার্ক নিরাপদ
4. সব পোর্ট খুলে দেওয়া

40. একটি Pod অন্য Namespace-এর Service-কে কীভাবে ডাকে? (*Beginner* • অধ্যায় 8)

1. শুধু Service-র নাম দিলেই সব Namespace থেকে চলে
2. IP ঠিকানা মুখস্থ করে
3. <service>.<namespace>.svc.cluster.local — সম্পূর্ণ DNS নাম দিয়ে
4. Pod-এর নাম দিয়ে

41. StatefulSet-এর বদলে Deployment দিয়ে ডেটাবেস চালানো কেন ঝুঁকিপূর্ণ? (*Intermediate* • অধ্যায় 9)

1. Deployment Pod-দের স্থিতিশীল নেটওয়ার্ক পরিচয় ও স্থিতিশীল স্টোরেজ দেয় না; Pod বদলালে ডেটা ও পরিচয় হারাতে পারে
2. Deployment ধীরগতির
3. Deployment-এ PVC ব্যবহার করা যায় না

4. Deployment শুধু stateless অ্যাপের জন্য তৈরি — তবে ডেটাবেসও stateless

42. CSI driver-এর কাজ কী? (*Beginner* • অধ্যায় 9)

1. Pod-এর নেটওয়ার্ক তৈরি করা
2. কনটেইনার image টানা
3. স্টোরেজ সিস্টেমকে Kubernetes-এর সাথে যুক্ত করা — volume তৈরি, সংযুক্তি ও snapshot পরিচালনা
4. সিক্রেট এনক্রিপ্ট করা

43. Volume-এর data হারিয়ে যাওয়ার সবচেয়ে সাধারণ ভুল কোনটি? (*Intermediate* • অধ্যায় 9)

1. PVC খুব বড় রাখা
2. StorageClass-এর নাম ভুল লেখা
3. Snapshot নেওয়া
4. Pod-এর সাথে বাঁধা ephemeral volume-কে স্থায়ী ডেটার জন্য ব্যবহার করা

44. TechNova-র PostgreSQL Pod নোড বদলালে ডেটা টিকে থাকবে কি না, তা কী নির্ধারণ করে? (*Intermediate* • অধ্যায় 9)

1. Pod-এর নাম
2. PVC-র access mode, StorageClass ও reclaim policy — volume Pod-এর জীবনচক্রের বাইরে টিকে কি না
3. Deployment-এর replica সংখ্যা
4. Ingress-র সেটিংস

45. StorageClass-এ volumeBindingMode: WaitForFirstConsumer ব্যবহারের সুবিধা কী? (*Advanced* • অধ্যায় 9)

1. Volume দ্রুত তৈরি হয়
2. Volume তখনই তৈরি হয় যখন Pod-এর নোড ঠিক হয় — ভুল availability zone-এ volume তৈরি হয় না
3. Volume ছোট হয়
4. Pod-এর প্রয়োজন হয় না

46. HPA কোন মেট্রিকের ভিত্তিতে replica বাড়ায়? (*Beginner* • অধ্যায় 10)

1. Pod-এর লগের আকার
2. নোডের সংখ্যা
3. দিনের সময়
4. CPU/memory ব্যবহার বা কাস্টম মেট্রিক — লক্ষ্যমাত্রার তুলনায় বর্তমান ব্যবহার

47. TechNova-র অ্যাপ সকালে হঠাৎ ট্রাফিক বাড়লে Pod বাড়তে দেরি হয়। সবচেয়ে সম্ভাব্য কারণ কোনটি? (*Intermediate* • অধ্যায় 10)

1. Pod-এর resource request বেশি, অথবা image টানতে দেরি হয়, বা HPA-র মেট্রিক সংগ্রহে বিলম্ব
2. HPA বন্ধ আছে — তবে replica বাড়ছে, শুধু ধীরে
3. Ingress খুব দ্রুত

4. ডেটাবেস খুব ভালো

48. Cluster Autoscaler ও HPA-র সম্পর্ক কী? (*Beginner* • অধ্যায় 10)

1. দুটো একই কাজ করে
2. Cluster Autoscaler HPA-কে বন্ধ করে দেয়
3. HPA Pod বাড়ায়; Pod বসানোর জায়গা না থাকলে Cluster Autoscaler নোড বাড়ায় — দুটি একসাথে কাজ করে
4. HPA নোড বাড়ায়, Cluster Autoscaler Pod বাড়ায়

49. Pod-এর resource request খুব কম ধরা হলে কী ঘটে? (*Intermediate* • অধ্যায় 10)

1. নোডে বেশি Pod বসে, চাপ বাড়লে CPU থ্রটলিং বা OOMKill হয়
2. Pod দ্রুত চলে
3. খরচ কমে, কোনো ঝুঁকি নেই
4. Scheduler Pod বসায় না

50. VPA ব্যবহারের প্রধান সতর্কতা কোনটি? (*Advanced* • অধ্যায় 10)

1. VPA কখনোই সুপারিশ দেয় না
2. VPA সুপারিশ প্রয়োগ করতে Pod পুনরায় চালু করে — প্রোডাকশনে হঠাৎ restart ব্যাঘাত ঘটাতে পারে
3. VPA শুধু নোডের আকার বদলায়
4. VPA HPA-র সাথে একসাথে চলতে পারে না, কখনোই

51. Canary deployment-এর মূল উদ্দেশ্য কী? (*Beginner* • অধ্যায় 11)

1. নতুন সংস্করণ দ্রুত সবাইকে দেওয়া
2. পুরোনো সংস্করণ মুছে ফেলা
3. নতুন সংস্করণে অল্প ট্রাফিক পাঠিয়ে ঝুঁকি যাচাই করা, তারপর ধাপে ধাপে বাড়ানো
4. ডেপ্লয় দ্রুত করা, পরীক্ষা ছাড়া

52. Argo Rollouts-এ analysis চলাকালে ত্রুটির হার বেড়ে গেলে কী হয়? (*Intermediate* • অধ্যায় 11)

1. Rollout আরও দ্রুত এগোয়
2. Analysis উপেক্ষা করা হয়
3. Pod-গুলো মুছে যায়
4. Rollout স্বয়ংক্রিয়ভাবে থেমে যায় বা আগের স্থিতিশীল সংস্করণে ফিরে যায়

53. Blue-green ও canary-র মধ্যে মূল পার্থক্য কী? (*Beginner* • অধ্যায় 11)

1. দুটো একই জিনিস
2. Blue-green-এ পুরো নতুন পরিবেশ তৈরি করে একসাথে সুইচ করা হয়; canary-তে ট্রাফিক ধাপে ধাপে ভাগ করা হয়
3. Blue-green শুধু ডেটাবেসের জন্য
4. Canary-তে পুরোনো সংস্করণ রাখা যায় না

54. Flagger ব্যবহারের সময় সবচেয়ে গুরুত্বপূর্ণ পূর্বশর্ত কোনটি? (*Advanced* • অধ্যায় 11)

1. অনেক replica থাকা
2. Ingress না থাকা
3. নির্ভরযোগ্য মেট্রিক ও স্বাস্থ্য-পরীক্ষা, যাতে স্বয়ংক্রিয় সিদ্ধান্ত ভুল না হয়
4. GitOps বন্ধ রাখা

55. TechNova-র canary-তে নতুন সংস্করণে 5% ট্রাফিক গেল, কিন্তু ত্রুটি ধরা পড়ল না — কারণ মেট্রিক সংগ্রহে বিলম্ব ছিল। শিক্ষা কী? (*Advanced* • অধ্যায় 11)

1. Canary বন্ধ করে দেওয়া
2. ট্রাফিক 100% করে দেওয়া
3. মেট্রিক আর না দেখা
4. Analysis-এর সময়কাল ও মেট্রিকের বিলম্ব বিবেচনা করে ধাপের সময় বাড়ানো এবং সতর্কতা (alert) যুক্ত করা

56. Backstage-এর মূল উদ্দেশ্য কী? (*Beginner* • অধ্যায় 12)

1. ডেভেলপারদের জন্য একটি কেন্দ্রীয় পোর্টাল — সার্ভিস ক্যাটালগ, টেমপ্লেট, ডকুমেন্টেশন ও সরঞ্জাম এক জায়গায়
2. Kubernetes ক্লাস্টার চালানো
3. সিক্রেট সংরক্ষণ করা
4. নেটওয়ার্ক নীতি লেখা

57. Software Template (scaffolder) ব্যবহারের সবচেয়ে বড় সুবিধা কোনটি? (*Intermediate* • অধ্যায় 12)

1. টেমপ্লেট কোড লেখা কমিয়ে দেয়, তবে মান নিশ্চিত করে না
2. টেমপ্লেট শুধু নাম দেয়
3. নতুন সার্ভিস সবসময় দলের মান অনুযায়ী — একই কাঠামো, CI, পরীক্ষা ও পর্যবেক্ষণসহ — তৈরি হয়
4. টেমপ্লেট ব্যবহার করলে review লাগে না

58. Service Catalog-এ একটি সার্ভিসের মালিক (owner) নথিভুক্ত করা কেন গুরুত্বপূর্ণ? (*Beginner* • অধ্যায় 12)

1. Catalog সুন্দর দেখায়
2. মালিক জানা থাকলে সার্ভিস দ্রুত চলে
3. এটি শুধু রিপোর্টের জন্য
4. ঘটনার সময় কাকে ডাকতে হবে, তা জানা যায় এবং দায়িত্ব অস্পষ্ট থাকে না

59. IDP বানাতে গিয়ে সবচেয়ে সাধারণ ভুল কোনটি? (*Advanced* • অধ্যায় 12)

1. ডেভেলপারদের না জিজ্ঞেস করে শুধু প্ল্যাটফর্ম দলের পছন্দে সরঞ্জাম বানানো
2. খুব কম সরঞ্জাম দেওয়া
3. ডকুমেন্টেশন লেখা
4. টেমপ্লেট বানানো

60. TechNova-র Backstage-এ একটি সার্ভিসের তথ্য পুরোনো। সবচেয়ে টেকসই প্রতিকার কী? (*Intermediate* • অধ্যায় 12)

1. সার্ভিসটি Catalog থেকে মুছে দেওয়া
2. Catalog তথ্য কোড থেকে (যেমন catalog-info.yaml) স্বয়ংক্রিয়ভাবে হালনাগাদ করা, হাতে সম্পাদনা নয়
3. কাউকে মনে করে হালনাগাদ করতে বলা
4. Backstage বন্ধ করে দেওয়া

61. Golden path বলতে কী বোঝায়? (*Beginner* • অধ্যায় 13)

1. একমাত্র পথ, যা থেকে সরে যাওয়া নিষিদ্ধ
2. সবচেয়ে দ্রুত পথ, মান যাচাই ছাড়া
3. একটি গোপন পথ, যা শুধু সিনিয়ররা জানেন
4. দলের অনুমোদিত, পরীক্ষিত ও সমর্থিত একটি প্রমিত পথ — নতুন সার্ভিস বানানো থেকে ডেপ্লয় পর্যন্ত

62. প্ল্যাটফর্ম দলের সাফল্য মাপার সবচেয়ে ভালো সূচক কোনটি? (*Intermediate* • অধ্যায় 13)

1. ডেভেলপারদের সময় বাঁচছে কি না ও নতুন সার্ভিস চালু করতে কত সময় লাগে
2. প্ল্যাটফর্ম দল কতগুলো টুল বানিয়েছে
3. প্ল্যাটফর্মের ডকুমেন্টেশন কত পাতা
4. প্ল্যাটফর্ম দলের সদস্য সংখ্যা

63. একটি দল golden path মানতে চাইছে না। প্ল্যাটফর্ম প্রকৌশলীর প্রথম পদক্ষেপ কী হওয়া উচিত? (*Advanced* • অধ্যায় 13)

1. জোর করে মানানো
2. কেন মানছে না, তা শোনা — path-টি তাদের সমস্যা সমাধান করছে কি না, তা বোঝা
3. Path-টি আরও কঠোর করা
4. দলটিকে প্ল্যাটফর্ম ব্যবহার থেকে বাদ দেওয়া

64. Platform as a product ধারণার মূল কথা কী? (*Intermediate* • অধ্যায় 13)

1. প্ল্যাটফর্ম বিক্রি করা
2. প্ল্যাটফর্মকে একবার বানিয়ে ফেলে রাখা
3. প্ল্যাটফর্মকে পণ্যের মতো ভাবা — ব্যবহারকারীর প্রয়োজন, ফিডব্যাক ও উন্নতির চক্র নিয়ে
4. প্ল্যাটফর্মের দাম নির্ধারণ করা

65. TechNova-র প্ল্যাটফর্মে একটি টুল কেউ ব্যবহার করছে না। সিদ্ধান্ত নেওয়ার আগে কী যাচাই করবেন? (*Intermediate* • অধ্যায় 13)

1. টুলটি আসল সমস্যা সমাধান করে কি না, ব্যবহার করা কতটা কঠিন, এবং বিকল্প কী আছে
2. টুলটির নাম বদলানো
3. টুলটি আরও প্রচার করা
4. টুলটি মুছে ফেলা

66. Enterprise CI/CD-এ trunk-based development-এর সুবিধা কী? (*Intermediate* • অধ্যায় 14)

1. বড় বড় পরিবর্তন একসাথে মার্জ করা সহজ হয়
2. শাখা কখনোই মুছতে হয় না
3. ছোট, ঘন ঘন মার্জ — দীর্ঘ শাখার মার্জ-সংঘাত ও পুরোনো কোডের ঝুঁকি কমে
4. পরীক্ষা ছাড়া মার্জ করা যায়

67. CI পাইপলাইনে একটি পরিবর্তন ব্যর্থ হলে সবচেয়ে ভালো অভ্যাস কোনটি? (*Beginner* • অধ্যায় 14)

1. ব্যর্থতা উপেক্ষা করে মার্জ করা
2. পাইপলাইন বন্ধ করে দেওয়া
3. পরীক্ষা মুছে ফেলা
4. ব্যর্থতা দ্রুত জানানো এবং মূল শাখা (main) সবসময় সবুজ রাখা — ভাঙা main-এ নতুন কাজ না তোলা

68. TechNova-র প্রোডাকশন ডেপ্লয়ে manual approval ধাপ আছে। এটি কখন যুক্তিসংগত? (*Intermediate* • অধ্যায় 14)

1. প্রতিটি ছোট পরিবর্তনে, যাতে গতি কমে
2. উচ্চ-ঝুঁকির পরিবর্তনে — যেমন ডেটাবেস মাইগ্রেশন বা নিয়ন্ত্রিত পরিবেশে — মানুষের পর্যালোচনা যোগ করলে
3. কখনোই নয় — approval সবসময় খারাপ
4. শুধু শুক্রবারে

69. CI-তে ক্যাশ ব্যবহারের প্রধান ঝুঁকি কী? (*Intermediate* • অধ্যায় 14)

1. ক্যাশ সবসময় গতি কমায়
2. পুরোনো ক্যাশ থেকে ভুল নির্ভরতা (dependency) ব্যবহার করে ভুলভাবে পাস করা
3. ক্যাশ ব্যবহার করলে পরীক্ষা চলে না
4. ক্যাশ শুধু খরচ বাড়ায়

70. একটি মনোরেপোতে CI-র সবচেয়ে বড় চ্যালেঞ্জ কোনটি? (*Advanced* • অধ্যায় 14)

1. ফাইল সংরক্ষণ করা
2. Git ব্যবহার করা
3. শাখা বানানো
4. কোন অংশ বদলেছে তা ধরে শুধু প্রভাবিত অংশের পরীক্ষা চালানো (affected testing)

71. Terraform-এ remote state ব্যবহারের প্রধান কারণ কী? (*Beginner* • অধ্যায় 15)

1. দলের সবাই একই state দেখতে পায় এবং state lock দিয়ে একসাথে পরিবর্তনের সংঘাত এড়ানো যায়
2. State ফাইল ছোট হয়
3. Terraform দ্রুত চলে
4. State-এ সিক্রেট নিরাপদে থাকে

72. Terraform module-এর সংস্করণ (version) পিন করা কেন গুরুত্বপূর্ণ? (*Intermediate* • অধ্যায় 15)

1. পিন করলে module দ্রুত চলে
2. পিন না করলে Terraform চলে না
3. Module-র নতুন সংস্করণ হঠাৎ আচরণ বদলে দিলে প্রোডাকশন ভাঙতে পারে — পিন করা পরিবর্তন নিয়ন্ত্রণ করে
4. পিন করলে খরচ কমে

73. terraform plan-এ একটি রিসোর্স ধ্বংস (destroy) দেখাচ্ছে, যা আপনি আশা করেননি। প্রথম করণীয় কী? (*Intermediate* • অধ্যায় 15)

1. Apply বন্ধ করে কেন ধ্বংস দেখাচ্ছে তা খুঁজে বের করা — নাম, count বা চলকের পরিবর্তন কি না
2. তবুও Apply করা, কারণ Plan সবসময় ঠিক
3. State ফাইল মুছে ফেলা
4. রিসোর্সটি হাতে আবার বানানো

74. TechNova-র Terraform কোডে সিক্রেট state ফাইলে প্লেইন টেক্সটে আছে। প্রতিকার কী? (*Advanced* • অধ্যায় 15)

1. State ফাইল Git-এ তুলে দেওয়া
2. State-কে এনক্রিপ্টেড remote backend-এ রাখা, সিক্রেটের জন্য ভল্ট ব্যবহার করা এবং state-এ সংবেদনশীল মান এড়ানো
3. সিক্রেট কোডে লেখা
4. State ফাইল মুছে ফেলা

75. Terraform workspaces বনাম আলাদা ডিরেক্টরি — কখন আলাদা ডিরেক্টরি ভালো? (*Advanced* • অধ্যায় 15)

1. সবসময় workspace ভালো
2. কখনোই আলাদা ডিরেক্টরি নয়
3. পরিবেশগুলোর কাঠামো ভিন্ন হলে বা আলাদা অনুমতি ও পর্যালোচনা চাইলে
4. শুধু ছোট প্রকল্পে

76. Ansible-এ idempotence কেন গুরুত্বপূর্ণ? (*Beginner* • অধ্যায় 16)

1. Idempotence গতি বাড়ায়
2. Idempotence ছাড়া Ansible চলে না
3. Idempotence শুধু নামের জন্য
4. একই playbook বারবার চালালেও দ্বিতীয়বার আর পরিবর্তন হয় না — নিরাপদে পুনরায় চালানো যায়

77. Ansible-এ inventory-র ভূমিকা কী? (*Beginner* • অধ্যায় 16)

1. Playbook-এর সোর্স কোড সংরক্ষণ করা
2. কোন হোস্টে কী চালাতে হবে এবং হোস্টগুলো কীভাবে গোষ্ঠীবদ্ধ, তা নির্ধারণ করা
3. সিক্রেট এনক্রিপ্ট করা
4. লগ সংরক্ষণ করা

78. শত শত সার্ভারে Ansible চালাতে ধীরগতি হলে প্রথম কৌশল কোনটি? (*Intermediate* • অধ্যায় 16)

1. সার্ভার কমানো
2. Playbook মুছে ফেলা
3. Fork বাড়ানো, কৌশল (strategy) যাচাই করা এবং gathering facts সীমিত করা
4. Ansible বন্ধ করা

79. Ansible Vault ব্যবহারের উদ্দেশ্য কী? (*Beginner* • অধ্যায় 16)

1. Playbook দ্রুত করা
2. সার্ভার এনক্রিপ্ট করা
3. Inventory লুকানো
4. সিক্রেট এনক্রিপ্ট করে Git-এ নিরাপদে রাখা

80. TechNova কনফিগারেশন ও provisioning দুটোই এক টুলে করতে চায়। সঠিক সিদ্ধান্ত কী? (*Advanced* • অধ্যায় 16)

1. দুটি কাজের জন্য আলাদা টুলের শক্তি বিবেচনা করা — Terraform provisioning-এ, Ansible কনফিগারেশনে শক্তিশালী; এক টুলে জোর করে দুটো করা প্রায়শই ভুল
2. একটি টুলেই সব করা উচিত, যেকোনো মূল্যে
3. দুটো টুল কখনোই একসাথে চলে না
4. শুধু Ansible দিয়ে সব করা যায়, Terraform লাগে না

81. Observability ও monitoring-এর পার্থক্য কী? (*Beginner* • অধ্যায় 17)

1. দুটো একই জিনিস
2. Monitoring নতুন, observability পুরোনো
3. Monitoring জানা প্রশ্নের উত্তর দেয়; observability অজানা প্রশ্নের উত্তর দেওয়ার ক্ষমতা দেয় — লগ, মেট্রিক ও ট্রেস একসাথে
4. Observability শুধু লগ দেখে

82. Distributed tracing-এ একটি trace কী দিয়ে গঠিত? (*Beginner* • অধ্যায় 17)

1. একটি লগ লাইন
2. একটি মেট্রিক
3. একটি ড্যাশবোর্ড
4. একাধিক span — প্রতিটি সার্ভিসে একটি কাজের অংশ, parent-child সম্পর্কে বাঁধা

83. OpenTelemetry-র সবচেয়ে বড় সুবিধা কোনটি? (*Intermediate* • অধ্যায় 17)

1. ভেন্ডর-নিরপেক্ষ (vendor-neutral) instrumentation — একটি SDK দিয়ে ডেটা যেকোনো backend-এ পাঠানো যায়
2. এটি সবচেয়ে দ্রুত লগ লেখে
3. এটি ডেটাবেস প্রতিস্থাপন করে
4. এটি কোড ছাড়াই কাজ করে

84. TechNova-র একটি অনুরোধ ধীর, কিন্তু কোন সার্ভিস ধীর তা জানা যাচ্ছে না। সবচেয়ে কার্যকর পদ্ধতি কোনটি? (*Intermediate* • অধ্যায় 17)

1. সব সার্ভিসের CPU বাড়ানো
2. Trace দেখে প্রতিটি span-এর সময় বিশ্লেষণ করা — কোন ধাপে সময় যাচ্ছে
3. লগে আরও লাইন যোগ করা
4. সার্ভার পুনরায় চালু করা

85. Cardinality বেশি এমন লেবেল (যেমন user ID) মেট্রিকে ব্যবহার করা কেন বিপজ্জনক? (*Advanced* • অধ্যায় 17)

1. মেট্রিক ধীর হয় না, তবে লগ বাড়ে
2. এটি নিরাপত্তা বাড়ায়
3. এটি ড্যাশবোর্ড সুন্দর করে
4. প্রতিটি নতুন মান নতুন time series তৈরি করে — স্টোরেজ ও খরচ বিস্ফোরিত হয়

86. 30 দিনের মাসে 99.99% SLO-র error budget কত? (*Beginner* • অধ্যায় 18)

1. 4 মিনিট 19 সেকেন্ড
2. 4 মিনিট 9 সেকেন্ড
3. 43 মিনিট 12 সেকেন্ড
4. 7 ঘণ্টা 18 মিনিট

87. Error budget শেষ হয়ে গেলে সঠিক সিদ্ধান্ত কোনটি? (*Intermediate* • অধ্যায় 18)

1. বাজেট বাড়িয়ে দেওয়া, যাতে ডেপ্লয় চলতে পারে
2. নতুন ফিচার ডেপ্লয় থামিয়ে নির্ভরযোগ্যতার কাজে মনোযোগ দেওয়া
3. SLO মুছে ফেলা
4. ত্রুটি লুকানো

88. SLI, SLO ও SLA-র পার্থক্য কী? (*Beginner* • অধ্যায় 18)

1. তিনটি একই জিনিস
2. SLI চুক্তি, SLA পরিমাপ
3. SLI পরিমাপ, SLO লক্ষ্য, SLA চুক্তিগত প্রতিশ্রুতি (প্রায়শই জরিমানাসহ)
4. SLO শুধু অভ্যন্তরীণ, SLI শুধু বাইরের

89. Burn rate alert ব্যবহারের সুবিধা কী? (*Advanced* • অধ্যায় 18)

1. Error budget কত দ্রুত শেষ হচ্ছে তা দেখে সমস্যা বড় হওয়ার আগেই সতর্ক করা যায়
2. এটি সব alert বন্ধ করে দেয়
3. এটি SLO বাড়িয়ে দেয়
4. এটি ত্রুটি কমিয়ে দেয়

90. TechNova 99.99% SLO বেছে নিল, কিন্তু খরচ তিনগুণ বেড়ে গেল। প্রকৌশলগত সিদ্ধান্ত কী হওয়া উচিত? (*Advanced* • অধ্যায় 18)

1. SLO আরও বাড়ানো
2. খরচ উপেক্ষা করা
3. ব্যবহারকারীর আসল প্রয়োজন যাচাই করে SLO পুনর্বিবেচনা করা — প্রতিটি অতিরিক্ত 9-এর খরচ ও সুবিধার হিসাব করা
4. SLO গোপন রাখা

91. OPA/Gatekeeper-এর মূল ভূমিকা কী? (*Beginner* • অধ্যায় 19)

1. Pod-এর image তৈরি করা
2. নেটওয়ার্ক ট্রাফিক রুট করা
3. সিক্রেট তৈরি করা
4. Admission স্তরে নীতি প্রয়োগ করা — নীতি-বিরুদ্ধ রিসোর্স ক্লাস্টারে ঢুকতে দেওয়া না

92. Kyverno ও OPA-র মধ্যে বেছে নেওয়ার সময় কী বিবেচনা করবেন? (*Intermediate* • অধ্যায় 19)

1. শুধু নামের জনপ্রিয়তা
2. দলের ভাষা-পছন্দ ও নীতির জটিলতা — Kyverno YAML-ভিত্তিক, OPA Rego-ভিত্তিক ও বেশি নমনীয়
3. শুধু দাম
4. শুধু ইনস্টলের গতি

93. Supply chain আক্রমণ থেকে বাঁচতে সবচেয়ে গুরুত্বপূর্ণ প্রথম পদক্ষেপ কোনটি? (*Advanced* • অধ্যায় 19)

1. Image-র নাম সুন্দর রাখা
2. Image স্বাক্ষর যাচাই (signing/verification) ও নির্ভরযোগ্য রেজিস্ট্রি থেকে শুধু অনুমোদিত image ব্যবহার করা
3. Image বড় রাখা
4. Image-র সংখ্যা বাড়ানো

94. TechNova-র একটি Pod-এ root ব্যবহারকারী চলছে। নীতি দিয়ে কীভাবে আটকাবেন? (*Intermediate* • অধ্যায় 19)

1. Pod-টি মুছে দেওয়া, নীতি ছাড়া
2. Root ব্যবহারকারীকে অনুমোদন দেওয়া
3. Pod-এর নাম বদলানো
4. Pod Security Standards / নীতি দিয়ে runAsNonRoot ও readOnlyRootFilesystem বাধ্যতামূলক করা

95. Admission controller ব্যর্থ হলে (fail-open বনাম fail-closed) সিদ্ধান্ত কীভাবে নেবেন? (*Advanced* • অধ্যায় 19)

1. নীতির ঝুঁকি দিয়ে — নিরাপত্তা-সংবেদনশীল নীতিতে fail-closed, কম ঝুঁকির নীতিতে সতর্কতার সাথে fail-open বিবেচনা
2. সবসময় fail-open

3. সবসময় fail-closed, প্রভাব না ভেবে
4. Admission বন্ধ করে দেওয়া

96. FinOps-এর মূল উদ্দেশ্য কী? (*Beginner* • অধ্যায় 20)

1. খরচ কমানো, যেকোনো মূল্যে
2. খরচ লুকানো
3. ক্লাউড খরচের দৃশ্যমানতা, দায়িত্ব ও নিয়ন্ত্রণ — খরচকে প্রকৌশল সিদ্ধান্তের অংশ করা
4. খরচ বাড়ানো

97. TechNova-র একটি Namespace-এ খরচ বেশি, কিন্তু কে দায়ী তা জানা যায় না। প্রথম পদক্ষেপ কী? (*Intermediate* • অধ্যায় 20)

1. Label ও ট্যাগ দিয়ে খরচ বরাদ্দ (allocation) করা — প্রতিটি Namespace, দল ও সার্ভিসের খরচ আলাদা দেখা
2. সব Pod মুছে ফেলা
3. খরচের রিপোর্ট বন্ধ করা
4. বাজেট বাড়ানো

98. Overprovisioned Pod-এর সবচেয়ে নির্ভরযোগ্য চিহ্ন কোনটি? (*Intermediate* • অধ্যায় 20)

1. Pod-এর সংখ্যা বেশি
2. Request করা resource-এর তুলনায় প্রকৃত ব্যবহার ধারাবাহিকভাবে অনেক কম
3. Pod-এর নাম দীর্ঘ
4. Pod দ্রুত চলে

99. Spot instance ব্যবহারের প্রধান ঝুঁকি কী? (*Intermediate* • অধ্যায় 20)

1. Spot সবসময় ধীর
2. Spot-এ Kubernetes চলে না
3. যেকোনো সময় instance ফিরিয়ে নেওয়া (preemption) হতে পারে — তাই শুধু সহনশীল (fault-tolerant) কাজে ব্যবহার করা উচিত
4. Spot-এ ডেটা রাখা যায় না

100. FinOps-এ showback বনাম chargeback-এর পার্থক্য কী? (*Advanced* • অধ্যায় 20)

1. দুটো একই জিনিস
2. Showback টাকা কাটে, chargeback দেখায়
3. Showback শুধু রিপোর্ট, chargeback শুধু সতর্কতা
4. Showback খরচ দেখায়, কিন্তু টাকা কাটে না; chargeback আসলেই দলের বাজেট থেকে খরচ কাটে

101. RTO ও RPO-র পার্থক্য কী? (*Beginner* • অধ্যায় 21)

1. দুটো একই জিনিস
2. RTO কতক্ষণে ফিরে আসতে হবে; RPO কতটুকু ডেটা হারানো সহ্য করা যায়
3. RTO ডেটা, RPO সময়

4. RTO শুধু নেটওয়ার্কের জন্য

102. Disaster recovery পরিকল্পনা কখন বিশ্বাসযোগ্য হয়? (*Intermediate* • অধ্যায় 21)

1. পরিকল্পনা লিখলেই
2. ব্যাকআপ নিলেই
3. নিয়মিত মহড়া (drill) দিয়ে পরীক্ষা করে — কাগজে লেখা পরিকল্পনা যথেষ্ট নয়
4. কেউ পরিকল্পনাটি পড়লেই

103. Chaos engineering-এর উদ্দেশ্য কী? (*Beginner* • অধ্যায় 21)

1. সিস্টেম ভাঙা, আনন্দের জন্য
2. প্রোডাকশনে বিশৃঙ্খলা তৈরি করা
3. সব সার্ভার বন্ধ করা
4. নিয়ন্ত্রিতভাবে ব্যর্থতা ঘটিয়ে সিস্টেমের দুর্বলতা আগেই খুঁজে বের করা

104. TechNova-র ব্যাকআপ আছে, কিন্তু পুনরুদ্ধার কখনো পরীক্ষা করা হয়নি। সবচেয়ে বড় ঝুঁকি কী? (*Intermediate* • অধ্যায় 21)

1. ব্যাকআপটি আসলে কাজ নাও করতে পারে — দূষিত, অসম্পূর্ণ বা ভুল পদ্ধতির হতে পারে
2. ব্যাকআপের আকার বড়
3. ব্যাকআপ নিতে সময় লাগে
4. ব্যাকআপের নাম ভুল

105. GameDay পরিচালনার সময় সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা ব্যবস্থা কী? (*Advanced* • অধ্যায় 21)

1. সবাইকে না জানিয়ে পরীক্ষা করা
2. প্রোডাকশনে সবকিছু বন্ধ করা
3. Blast radius সীমিত রাখা ও দ্রুত থামানোর (abort) ব্যবস্থা — প্রয়োজনে পরীক্ষা বন্ধ করা যায়
4. পরীক্ষার সময় কাউকে না ডাকা

106. Multi-cloud কৌশলের সবচেয়ে বড় পরিচালন-খরচ কোনটি? (*Intermediate* • অধ্যায় 22)

1. ক্লাউডের নাম মনে রাখা
2. একটি ক্লাউডে অ্যাকাউন্ট খোলা
3. DNS রেকর্ড লেখা
4. প্রতিটি ক্লাউডের আলাদা API, পরিচয়, নেটওয়ার্ক ও খরচের মডেল সামলানো

107. Hybrid cloud-এ সবচেয়ে সংবেদনশীল সংযোগ কোনটি? (*Beginner* • অধ্যায় 22)

1. অন-প্রিমিসিস ও ক্লাউডের মধ্যে নেটওয়ার্ক সংযোগ — লেটেন্সি, নিরাপত্তা ও নির্ভরযোগ্যতা
2. অফিসের Wi-Fi
3. ডেভেলপারের ল্যাপটপ
4. প্রিন্টার সংযোগ

108. Vendor lock-in কমানোর সবচেয়ে বাস্তবসম্মত উপায় কোনটি? (*Intermediate* • অধ্যায় 22)

1. কোনো ক্লাউডই ব্যবহার না করা
2. খোলা মান (open standards) ও পোর্টেবল সরঞ্জাম — Kubernetes, Terraform, OpenTelemetry — ব্যবহার করা
3. প্রতিদিন ক্লাউড বদলানো
4. সব ক্লাউডে একই জিনিস চালানো, খরচ না ভেবে

109. TechNova একটি ক্লাউডের বিশেষ সেবা (managed service) ব্যবহার করতে চায়। সিদ্ধান্তের ভারসাম্য কী? (*Advanced* • অধ্যায় 22)

1. Managed service সবসময় খারাপ
2. Managed service সবসময় ভালো
3. সিদ্ধান্ত নেওয়ার প্রয়োজন নেই
4. Managed service-এর গতি ও সুবিধা বনাম lock-in ও খরচ — কোনটি ব্যবসার জন্য গুরুত্বপূর্ণ, তা মাপা

110. Multi-cloud-এ পরিচয় (identity) পরিচালনার সবচেয়ে নিরাপদ পদ্ধতি কোনটি? (*Advanced* • অধ্যায় 22)

1. কেন্দ্রীয় পরিচয় প্রদানকারী (IdP) থেকে federation — প্রতিটি ক্লাউডে আলাদা ব্যবহারকারী না বানানো
2. প্রতিটি ক্লাউডে আলাদা পাসওয়ার্ড রাখা
3. একটি পাসওয়ার্ড সব জায়গায় ব্যবহার করা
4. পরিচয় ছাড়া চালানো

111. Zero-trust-এর মূল নীতি কী? (*Beginner* • অধ্যায় 23)

1. ভেতরের নেটওয়ার্ক সম্পূর্ণ নিরাপদ
2. কাউকে স্বয়ংক্রিয়ভাবে বিশ্বাস নয় — প্রতিটি অনুরোধ যাচাই করা, নেটওয়ার্কের ভেতরেও
3. ফায়ারওয়ালই যথেষ্ট
4. পাসওয়ার্ডই যথেষ্ট

112. Kubernetes Secret-কে Git-এ প্লেইন টেক্সটে রাখা কেন বিপজ্জনক? (*Beginner* • অধ্যায় 23)

1. Secret খুব বড় হয়
2. Secret Git-এ যায় না
3. Secret base64-এ থাকে, এনক্রিপ্টেড নয় — যে কেউ ডিকোড করতে পারে
4. Secret কখনোই ফাঁস হয় না

113. External Secrets Operator-এর সুবিধা কী? (*Intermediate* • অধ্যায় 23)

1. ভল্টের মতো বাইরের উৎস থেকে সিক্রেট Kubernetes Secret-এ সিঙ্ক করা — সিক্রেট Git-এ থাকে না
2. সিক্রেট মুছে ফেলা
3. সিক্রেট Git-এ লেখা
4. সিক্রেট ছাড়া চালানো

114. Workload identity ব্যবহারের প্রধান সুবিধা কী? (*Advanced* • অধ্যায় 23)

1. Pod দ্রুত চলে
2. Pod-এর সংখ্যা বাড়ে
3. Pod-কে ক্লাউড পরিচয় দেওয়া যায়, দীর্ঘমেয়াদি চাবি (key) ছাড়া — চাবি ফাঁসের ঝুঁকি কমে
4. চাবি আর কখনো লাগে না, কোনো যাচাই ছাড়া

115. TechNova-র একটি সিক্রেট ফাঁস হয়ে গেছে বলে সন্দেহ। প্রথম করণীয় কী? (*Intermediate* • অধ্যায় 23)

1. অপেক্ষা করা — হয়তো কেউ দেখেনি
2. সিক্রেটটি আবার একই মান দিয়ে তৈরি করা
3. ঘটনাটি লুকানো
4. সিক্রেটটি অবিলম্বে বাতিল (revoke) ও ঘোরানো (rotate) এবং কোথায় ব্যবহৃত হয়েছে তা খুঁজে বের করা

116. Enterprise transformation-এ সবচেয়ে সাধারণ ব্যর্থতার কারণ কোনটি? (*Advanced* • অধ্যায় 24)

1. খুব কম টুল কেনা
2. শুধু টুল বদলানো, কিন্তু প্রক্রিয়া, দায়িত্ব ও সংস্কৃতি না বদলানো
3. খুব বেশি ডকুমেন্টেশন
4. খুব ধীরে কাজ করা

117. Platform adoption মাপার সবচেয়ে অর্থপূর্ণ সূচক কোনটি? (*Intermediate* • অধ্যায় 24)

1. প্ল্যাটফর্মের বিজ্ঞাপন কতবার দেখানো হয়েছে
2. স্বেচ্ছায় ব্যবহারকারীর সংখ্যা ও তাদের সময় বাঁচা — জোর করে ব্যবহার নয়
3. প্ল্যাটফর্ম দলের বাজেট
4. প্ল্যাটফর্মের নাম কত সুন্দর

118. একটি বড় সংস্থায় DevOps transformation শুরু করার সবচেয়ে নিরাপদ পদ্ধতি কোনটি? (*Intermediate* • অধ্যায় 24)

1. একসাথে সব দলে পরিবর্তন চাপানো
2. কোনো পরিকল্পনা ছাড়া শুরু করা
3. শুধু টুল কিনে দেওয়া
4. একটি ছোট দলে পাইলট করে শেখা, তারপর প্রমাণসহ ছড়িয়ে দেওয়া

119. TechNova-র capstone প্রকল্পে সবচেয়ে গুরুত্বপূর্ণ মূল্যায়নের দিক কোনটি? (*Advanced* • অধ্যায় 24)

1. সিদ্ধান্তের যুক্তি রক্ষা করা — কেন এই নকশা, কী বাদ দেওয়া হলো, ঝুঁকি কী — শুধু কাজ করলেই নয়
2. কতগুলো টুল ব্যবহার করা হয়েছে
3. প্রেজেন্টেশন কত সুন্দর
4. কোড কত লাইন

120. Transformation-এ technical debt মোকাবিলার সবচেয়ে টেকসই কৌশল কোনটি? (*Intermediate* • অধ্যায় 24)

1. Debt উপেক্ষা করা
2. একবারে সব debt শোধ করা
3. প্রতিটি স্প্রিন্টে debt-এর জন্য নির্দিষ্ট সময় রাখা এবং debt-কে দৃশ্যমান (backlog-এ) করা
4. Debt লুকানো

মূল্যায়ন • ৩০টি প্রশ্ন

Short-Answer Questions — ৩০টি প্রশ্ন

প্রতিটি উত্তর ৩–৫ বাক্যে, উদাহরণসহ লিখুন। সংক্ষিপ্ত মডেল উত্তর Answer Guide-2-এ আছে।

1. **Control plane-এর একটি উপাদান ব্যর্থ হলে চলমান Pod-গুলোর কী হয় এবং কেন?** (*Beginner* • অধ্যায় ১)
2. **etcd snapshot নেওয়ার সঠিক সময় ও পদ্ধতি ব্যাখ্যা করুন।** (*Intermediate* • অধ্যায় ২)
3. **Multi-cluster পরিবেশে blast radius সীমিত করার তিনটি উপায় লিখুন।** (*Intermediate* • অধ্যায় ৩)
4. **GitOps-এ drift বলতে কী বোঝায়? Argo CD কীভাবে তা ধরে?** (*Beginner* • অধ্যায় ৪)
5. **Flux-এ reconciliation loop কীভাবে কাজ করে?** (*Beginner* • অধ্যায় ৫)
6. **Helm values ও Kustomize overlay-এর মধ্যে কখন কোনটি বেছে নেবেন? যুক্তিসহ লিখুন।** (*Intermediate* • অধ্যায় ৬)
7. **Service mesh ছাড়া mTLS চালু করার বিকল্প কী কী?** (*Intermediate* • অধ্যায় ৭)
8. **Gateway API কেন Ingress-এর চেয়ে ভূমিকা-ভিত্তিক নকশায় শক্তিশালী?** (*Advanced* • অধ্যায় ৮)
9. **StatefulSet কেন ডেটাবেসের জন্য উপযুক্ত? তিনটি কারণ দিন।** (*Beginner* • অধ্যায় ৯)
10. **HPA, VPA ও Cluster Autoscaler-এর দায়িত্ব আলাদা করে লিখুন।** (*Beginner* • অধ্যায় ১০)
11. **Canary ও blue-green deployment-এর ঝুঁকি ও খরচের তুলনা করুন।** (*Intermediate* • অধ্যায় ১১)
12. **Backstage catalog-এ owner ক্ষেত্রটি কেন incident response-এ গুরুত্বপূর্ণ?** (*Beginner* • অধ্যায় ১২)
13. **Golden path ও paved road ধারণার পার্থক্য ব্যাখ্যা করুন।** (*Advanced* • অধ্যায় ১৩)
14. **Trunk-based development-এ দীর্ঘ শাখার ঝুঁকি কী?** (*Beginner* • অধ্যায় ১৪)
15. **Terraform state lock কেন প্রয়োজন? Lock ছাড়া কী ঘটতে পারে?** (*Intermediate* • অধ্যায় ১৫)
16. **Ansible-এ idempotent task লেখার একটি উদাহরণ দিন।** (*Beginner* • অধ্যায় ১৬)
17. **Trace, span ও log-এর সম্পর্ক ব্যাখ্যা করুন।** (*Beginner* • অধ্যায় ১৭)
18. **99.9% ও 99.99% SLO-র মধ্যে 30 দিনে error budget-এর পার্থক্য হিসাবসহ দেখান।** (*Intermediate* • অধ্যায় ১৮)
19. **Policy as Code বলতে কী বোঝায়? একটি উদাহরণ দিন।** (*Beginner* • অধ্যায় ১৯)
20. **FinOps-এ ট্যাগিং কৌশল না থাকলে কী সমস্যা হয়?** (*Beginner* • অধ্যায় ২০)
21. **RTO ও RPO দিয়ে একটি DR পরিকল্পনার লক্ষ্য কীভাবে লিখবেন? উদাহরণ দিন।** (*Intermediate* • অধ্যায় ২১)
22. **Multi-cloud-এ abstraction layer-এর সুবিধা ও খরচ লিখুন।** (*Advanced* • অধ্যায় ২২)
23. **Kubernetes Secret কেন এনক্রিপশন নয়? নিরাপদ বিকল্প কী?** (*Intermediate* • অধ্যায় ২৩)
24. **Transformation-এ পাইলট দল বেছে নেওয়ার মানদণ্ড কী হওয়া উচিত?** (*Advanced* • অধ্যায় ২৪)
25. **Argo CD-তে App of Apps প্যাটার্ন কখন ব্যবহার করবেন?** (*Advanced* • অধ্যায় ৪)
26. **Progressive delivery-তে analysis template-এ ভুল মেট্রিক থাকলে কী ঝুঁকি হয়?** (*Advanced* • অধ্যায় ১১)
27. **OpenTelemetry collector ব্যবহারের সুবিধা কী?** (*Intermediate* • অধ্যায় ১৭)
28. **Chaos experiment-এর blast radius সীমিত করার তিনটি উপায় লিখুন।** (*Advanced* • অধ্যায় ২১)
29. **IDP-র সাফল্য মাপার তিনটি সূচক প্রস্তাব করুন।** (*Intermediate* • অধ্যায় ১২–১৩)
30. **Enterprise CI/CD-এ main শাখা ভেঙে গেলে দলের প্রথম করণীয় কী?** (*Beginner* • অধ্যায় ১৪)

মূল্যায়ন • ২০টি দৃশ্যকল্প

Advanced Troubleshooting Scenarios — ২০টি দৃশ্যকল্প

প্রতিটি দৃশ্যকল্পে পরিস্থিতি, লক্ষণ ও প্রশ্ন দেওয়া আছে। উত্তরে প্রথমে আপনার অনুমান ও যাচাইয়ের ক্রম লিখুন, তারপর করণীয়।

1. Argo CD ক্রমাগত OutOfSync

পরিস্থিতি: TechNova-র প্রোডাকশন অ্যাপ Argo CD-তে OutOfSync দেখাচ্ছে। Git-এ শেষ কমিট ৩ দিন আগের, কিন্তু কেউ ক্লাস্টারে হাতে কিছু বদলায়নি বলে দাবি করছে। Sync করলে কয়েক মিনিট পর আবার OutOfSync হয়।

লক্ষণ: Argo CD UI-তে diff-এ একটি ConfigMap-এর মান বারবার বদলে যাচ্ছে; অ্যাপটি নিজেই ConfigMap আপডেট করে বলে সন্দেহ।

প্রশ্ন: (১) Diff-টি আসলে কী বদলাচ্ছে, তা কীভাবে নিশ্চিত করবেন? (২) অ্যাপ নিজে রিসোর্স বদলালে GitOps মডেলে কী করবেন — ignoreDifferences, নাকি অ্যাপের নকশা বদলাবেন? যুক্তি দিন। (৩) Self-heal চালু করলে কী হবে, আর কেন তা এখানে সমাধান নাও হতে পারে?

2. etcd ডিস্ক পূর্ণ

পরিস্থিতি: সকাল ৯টায় API server-এর লেটেন্সি হঠাৎ ৩০ সেকেন্ড ছাড়িয়ে গেল। etcd-র ডিস্ক ব্যবহার 98%। ক্লাস্টারে নতুন Pod বসছে না, কিন্তু চলমান Pod চলছে।

লক্ষণ: kubectl আদেশ টাইমআউট হচ্ছে; etcd লগে 'no space left on device' ও compaction সংক্রান্ত সতর্কতা।

প্রশ্ন: (১) এখনই সবচেয়ে নিরাপদ তাৎক্ষণিক পদক্ষেপ কী? (২) কেন শুধু ডিস্ক বাড়ানোই যথেষ্ট নয় — compaction ও ইতিহাস (revision) ব্যবস্থাপনা কীভাবে ঠিক করবেন? (৩) ভবিষ্যতে এই পরিস্থিতি আগেই ধরার জন্য কোন alert বসাবেন?

3. Canary-তে ধীরে ধীরে বাড়ছে ত্রুটি

পরিস্থিতি: নতুন রিলিজ canary-তে 10% ট্রাফিক পাচ্ছে। ত্রুটির হার 0.2% থেকে ধীরে ধীরে 1.5% হচ্ছে, কিন্তু analysis gate এখনও সবুজ।

লক্ষণ: ত্রুটি শুধু একটি নির্দিষ্ট অঞ্চলের (region) ব্যবহারকারীদের মধ্যে; মেট্রিক সংগ্রহে ৫ মিনিটের বিলম্ব আছে।

প্রশ্ন: (১) Gate সবুজ থাকলেও আপনি কি rollout থামাবেন? যুক্তি দিন। (২) মেট্রিক বিলম্ব analysis-কে কীভাবে ভুল পথে নেয়? (৩) পরের rollout-এ কোন গার্ডরেল যোগ করবেন?

4. একটি Namespace-এ খরচ বিস্ফোরণ

পরিস্থিতি: মাসের ১০ তারিখে FinOps ড্যাশবোর্ডে দেখা গেল একটি Namespace-এর খরচ গত মাসের পুরো খরচ ছাড়িয়ে গেছে। কোনো নতুন ডেপ্লয় ঘোষণা হয়নি।

লক্ষণ: একটি CronJob প্রতি মিনিটে বড় Pod চালাচ্ছে; resource request খুব বেশি, কিন্তু প্রকৃত ব্যবহার কম।

প্রশ্ন: (১) খরচের উৎস কীভাবে দ্রুত চিহ্নিত করবেন? (২) তাৎক্ষণিকভাবে খরচ থামাতে কী করবেন — আর কী করবেন না? (৩) ভবিষ্যতে এই ধরনের বিস্ফোরণ রোধে কোন নীতি (policy) বসাবেন?

5. Service mesh-এ হঠাৎ mTLS ব্যর্থতা

পরিস্থিতি: গত রাতে Istio আপগ্রেডের পর দুটি সার্ভিসের মধ্যে যোগাযোগ ব্যর্থ হচ্ছে। একটি সার্ভিসে sidecar আছে, অন্যটিতে নতুন Pod-এ sidecar ইনজেক্ট হয়নি।

লক্ষণ: ত্রুটি: 'connection reset by peer'; PeerAuthentication STRICT মোডে আছে।

প্রশ্ন: (১) STRICT মোডে একটি পক্ষে sidecar না থাকলে কী হয়? (২) তাৎক্ষণিকভাবে সেবা ফেরাতে কী করবেন — মোড বদলাবেন, নাকি sidecar ঠিক করবেন? ঝুঁকিসহ যুক্তি দিন। (৩) আপগ্রেডের আগে কী পরীক্ষা করলে এটি ধরা পড়ত?

6. Helm আপগ্রেড আটকে গেল

পরিস্থিতি: helm upgrade ব্যর্থ হয়ে release 'pending-upgrade' অবস্থায় আটকে আছে। আর কোনো upgrade বা rollback চলছে না।

লক্ষণ: helm history-তে শেষ revision ব্যর্থ; একটি Secret (release তথ্য) নিয়ে সন্দেহ।

প্রশ্ন: (১) Release-এর অবস্থা কীভাবে নিরাপদে পরীক্ষা করবেন? (২) Rollback বনাম হাতে Secret সম্পাদনা — কোনটি নিরাপদ এবং কেন? (৩) ভবিষ্যতে আটকে যাওয়া এড়াতে কী অভ্যাস নেবেন?

7. Multi-cluster failover পরীক্ষায় ব্যর্থতা

পরিস্থিতি: TechNova-র DR মহড়ায় প্রাথমিক ক্লাস্টার বন্ধ করা হলো, কিন্তু ট্রাফিক দ্বিতীয় ক্লাস্টারে গেল না। DNS-এ রেকর্ড আছে, কিন্তু স্বাস্থ্য-পরীক্ষা (health check) প্রাথমিক ক্লাস্টারকেই সুস্থ দেখাচ্ছে।

লক্ষণ: Global Load Balancer-এর স্বাস্থ্য-পরীক্ষা একটি ক্যাশ করা endpoint দেখছে; দ্বিতীয় ক্লাস্টারে ডেটা ২ ঘণ্টা পুরোনো।

প্রশ্ন: (১) Failover কেন হলো না — স্বাস্থ্য-পরীক্ষার নকশায় কী ভুল? (২) দ্বিতীয় ক্লাস্টারের ডেটা পুরোনো হলে failover করলেও কী ঝুঁকি থাকে? (৩) পরের মহড়ার আগে কোন তিনটি জিনিস যাচাই করবেন?

8. HPA স্কেল করছে না

পরিস্থিতি: ট্রাফিক তিনগুণ বেড়েছে, কিন্তু replica সংখ্যা একই আছে। HPA-র target CPU 50% দেখাচ্ছে, অথচ Pod-গুলোর CPU 95%।

লক্ষণ: Metrics Server থেকে মেট্রিক আসছে, কিন্তু Pod-এর resource request খুব কম ধরা হয়েছে।

প্রশ্ন: (১) Target 50% দেখালেও CPU 95% কেন — request বনাম ব্যবহারের পার্থক্য ব্যাখ্যা করুন। (২) সঠিক প্রতিকার কী — HPA target বদলানো, নাকি request ঠিক করা? (৩) এই ভুল আবার না হতে কী করবেন?

9. Stateful ডেটাবেস Pod নোড বদলাল

পরিস্থিতি: PostgreSQL Pod একটি নোড থেকে অন্য নোডে গেল। Pod চলছে, কিন্তু ডেটা গতকালের — আজকের লেনদেন নেই।

লক্ষণ: PVC-র volumeBindingMode Immediate ছিল; নতুন নোড ভিন্ন availability zone-এ।

প্রশ্ন: (১) ডেটা কেন পুরোনো — volume কীভাবে ভুল zone-এ বাঁধা পড়ল? (২) এখন ডেটা উদ্ধারের সম্ভাব্য পথ কী? (৩) WaitForFirstConsumer কীভাবে এই সমস্যা এড়াত?

10. GitOps-এ ভুল কনফিগারেশন প্রোডাকশনে গেল

পরিস্থিতি: একজন ডেভেলপার ভুল করে replica সংখ্যা 3-এর বদলে 300 লিখে মার্জ করলেন। Automated sync তা প্রোডাকশনে প্রয়োগ করল; ক্লাস্টারের resource শেষ।

লক্ষণ: PR-এ কেউ সংখ্যাটি খেয়াল করেনি; কোনো নীতি-পরীক্ষা ছিল না।

প্রশ্ন: (১) তাৎক্ষণিকভাবে কীভাবে থামাবেন ও আগের অবস্থায় ফেরাবেন? (২) এই ভুল আবার না হতে PR স্তরে কোন তিনটি নিয়ন্ত্রণ যোগ করবেন? (৩) Automated sync কি এখানে বন্ধ করা উচিত — যুক্তিসহ মত দিন।

11. Observability-তে অন্ধকার সময়

পরিস্থিতি: একটি অনুরোধ ধীর, কিন্তু কোনো trace পাওয়া যাচ্ছে না। লগে অনুরোধটি আছে, মেট্রিকে লেটেন্সি বাড়ছে।

লক্ষণ: Sampling হার 1% — ধীর অনুরোধগুলো sample-এ পড়ছে না; একটি সার্ভিসে instrumentation নেই।

প্রশ্ন: (১) Sampling কীভাবে ধীর অনুরোধ লুকিয়ে ফেলছে? (২) Tail-based sampling কীভাবে সাহায্য করবে? (৩) Instrumentation-বিহীন সার্ভিসটি কীভাবে শনাক্ত করবেন?

12. Policy হঠাৎ সব ডেপ্লয় আটকে দিল

পরিস্থিতি: নতুন OPA নীতি চালুর পর সব Pod তৈরি বন্ধ — এমনকি আগে চলা অ্যাপের আপডেটও।

লক্ষণ: নীতিটি runAsNonRoot বাধ্যতামূলক করেছে, কিন্তু একটি পুরোনো অ্যাপ root-এ চলে।

প্রশ্ন: (১) নীতিটি কি ভুল, নাকি প্রয়োগের পদ্ধতি ভুল — কীভাবে ঠিক করবেন? (২) Audit (dry-run) মোড কীভাবে সাহায্য করত? (৩) পুরোনো অ্যাপটির জন্য ব্যতিক্রম (exception) দেওয়ার নিরাপদ পদ্ধতি কী?

13. Terraform state নষ্ট

পরিস্থিতি: দুজন প্রকৌশলী একই সময়ে terraform apply চালালেন। State ফাইল এখন অসঙ্গত; Plan-এ শত শত পরিবর্তন দেখাচ্ছে।

লক্ষণ: Remote backend-এ lock চালু ছিল না; একজন local state ব্যবহার করছিলেন।

প্রশ্ন: (১) এখন Apply করা কেন বিপজ্জনক? (২) State পুনরুদ্ধারের ধাপগুলো কী — ব্যাকআপ, import, refresh-এর ভূমিকা কী? (৩) ভবিষ্যতে একসাথে Apply কীভাবে আটকাবেন?

14. সিক্রেট ফাঁসের সন্দেহ

পরিস্থিতি: একটি API কী GitHub-এ একটি public রিপোজিটরিতে ভুল করে কমিট হয়েছে। কীটি প্রোডাকশন ডেটাবেসে ব্যবহৃত হয়।

লক্ষণ: কমিটটি ২ ঘণ্টা আগের; কীটি এখনও সক্রিয়।

প্রশ্ন: (১) প্রথম ৩০ মিনিটে কী কী করবেন — ক্রমানুসারে লিখুন। (২) শুধু কমিট মুছলেই কি যথেষ্ট? কেন নয়? (৩) ভবিষ্যতে ফাঁস রোধে কোন দুটি নিয়ন্ত্রণ বসাবেন?

15. Cluster upgrade-এ Pod ব্যাহত

পরিস্থিতি: Kubernetes সংস্করণ আপগ্রেডের সময় অনেক Pod একসাথে বন্ধ হয়ে সেবা ব্যাহত হলো।

লক্ষণ: PodDisruptionBudget ছিল না; নোড drain একসাথে অনেক নোডে চলেছে।

প্রশ্ন: (১) Drain কীভাবে সেবা ব্যাহত করল? (২) PDB ও ধাপে ধাপে drain কীভাবে সাহায্য করত? (৩) আপগ্রেডের আগে কোন পরীক্ষা (pre-flight) করবেন?

16. Backstage-এ কেউ নতুন সার্ভিস বানাচ্ছে না

পরিস্থিতি: IDP চালুর তিন মাস পরও দলগুলো হাতে সার্ভিস বানাচ্ছে; টেমপ্লেট ব্যবহার হচ্ছে না।

লক্ষণ: টেমপ্লেটটি দলের প্রয়োজন মেটায় না; টেমপ্লেটে পুরোনো লাইব্রেরি সংস্করণ আছে।

প্রশ্ন: (১) ব্যবহার না হওয়ার কারণ কীভাবে খুঁজে বের করবেন? (২) টেমপ্লেট ঠিক করার আগে কার সাথে কথা বলবেন? (৩) সাফল্য মাপার সূচক কী হবে?

17. DR ব্যাকআপ পুনরুদ্ধার ব্যর্থ

পরিস্থিতি: মহড়ায় ব্যাকআপ থেকে ডেটাবেস পুনরুদ্ধার করতে গিয়ে দেখা গেল ব্যাকআপ ফাইলটি দূষিত। শেষ সফল পুনরুদ্ধার পরীক্ষা ৮ মাস আগের।

লক্ষণ: ব্যাকআপ প্রতিদিন নেওয়া হচ্ছিল, কিন্তু কেউ যাচাই করেনি।

প্রশ্ন: (১) ব্যাকআপ নেওয়া হলেও পুনরুদ্ধার ব্যর্থ হওয়ার কারণ কী হতে পারে? (২) এখনই কী করবেন — পুরোনো ব্যাকআপ, নাকি অন্য উৎস? (৩) ব্যাকআপ যাচাইয়ের নিয়মিত প্রক্রিয়া কেমন হবে?

18. Ingress-এ হঠাৎ 502

পরিস্থিতি: একটি সার্ভিসে নতুন সংস্করণ ডেপ্লয়ের পর Ingress থেকে 502 আসছে। Pod চলছে, কিন্তু readiness probe ব্যর্থ।

লক্ষণ: নতুন সংস্করণের startup সময় ৯০ সেকেন্ড, কিন্তু probe-এর timeout ৫ সেকেন্ড।

প্রশ্ন: (১) 502-এর আসল কারণ কী — Pod, নাকি probe-এর সেটিংস? (২) তাৎক্ষণিক প্রতিকার ও স্থায়ী প্রতিকার আলাদা করে লিখুন। (৩) Rollout-এর আগে কীভাবে এই সমস্যা ধরা যেত?

19. দুটি ক্লাস্টারে নীতির অমিল

পরিস্থিতি: Audit-এ দেখা গেল ঢাকা ক্লাস্টারে Pod Security নীতি আছে, চট্টগ্রাম ক্লাস্টারে নেই। একই অ্যাপ দুই জায়গায় ভিন্ন আচরণ করছে।

লক্ষণ: নীতি হাতে প্রয়োগ করা হয়েছিল, GitOps-এ ছিল না।

প্রশ্ন: (১) অমিলটি কীভাবে তৈরি হলো? (২) দুটি ক্লাস্টারে নীতি এক রাখার টেকসই পদ্ধতি কী? (৩) Audit-এ এই অমিল আগেই ধরার উপায় কী?

20. Chaos পরীক্ষা সীমা ছাড়িয়ে গেল

পরিস্থিতি: একটি chaos পরীক্ষা ভুল করে প্রোডাকশনের ডেটাবেস Pod বন্ধ করে দিল। পরীক্ষাটি staging-এর জন্য ছিল।

লক্ষণ: পরীক্ষার target selector ভুল ছিল; abort বোতাম কাজ করেনি বলে অভিযোগ।

প্রশ্ন: (১) পরীক্ষাটি কীভাবে ভুল পরিবেশে গেল — selector ও পরিবেশ আলাদা করার ব্যর্থতা কোথায়? (২) Abort ব্যবস্থা পরীক্ষা না করে চালানো কেন ভুল? (৩) ভবিষ্যতে chaos পরীক্ষার অনুমোদন প্রক্রিয়া কেমন হবে?

মূল্যায়ন • ১০টি কেস

Enterprise Production Incident Cases — ১০টি কেস

প্রতিটি কেস একটি বাস্তবসম্মত প্রোডাকশন ঘটনা। টাইমলাইনের ইঙ্গিত থেকে ব্যর্থতার শৃঙ্খল বের করুন, তারপর প্রশ্নগুলোর উত্তর দিন।

1. TechNova-র Black Friday বিভ্রাট

ঘটনা: TechNova-র e-commerce অ্যাপ Black Friday-তে হঠাৎ ধীর হয়ে গেল। সকাল ১০টায় ট্রাফিক স্বাভাবিকের পাঁচগুণ। HPA replica বাড়াল, কিন্তু নতুন Pod-গুলো বসতে পারল না — নোডে জায়গা নেই। Cluster Autoscaler নোড বাড়াল, কিন্তু নতুন নোড আসতে ৬ মিনিট লাগল; ততক্ষণে অনেক অনুরোধ টাইমআউট। ডেটাবেসের সংযোগ (connection) শেষ হয়ে গেল, কারণ প্রতিটি Pod অনেক সংযোগ খুলছিল।

টাইমলাইন: ০৯:৫৮ ট্রাফিক বাড়তে শুরু; ১০:০২ HPA replica 10 থেকে 40-এ; ১০:০৩ Pod Pending; ১০:০৫ Cluster Autoscaler নোড যোগ করল; ১০:০৯ নতুন নোড প্রস্তুত; ১০:১২ ডেটাবেস সংযোগ শেষ; ১০:৩০ হাতে replica কমিয়ে সেবা ফিরল।

প্রশ্ন: (১) ব্যর্থতার শৃঙ্খলটি (chain) চিহ্নিত করুন — কোন ঘটনা কোনটিকে ডেকে আনল? (২) নোড আসার ৬ মিনিটের ফাঁকটি কীভাবে সামলানো যেত — overprovisioning, priority, নাকি আগাম স্কেলিং? (৩) ডেটাবেস সংযোগের সমস্যাটি অ্যাপ স্তরে কীভাবে ঠিক করবেন? (৪) পরের বড় ট্রাফিকের আগে কোন তিনটি প্রস্তুতি নেবেন?

2. GitOps-এ ভুল image প্রোডাকশনে

ঘটনা: একটি Flux Image Automation নিয়ম রাত ২টায় নতুন image ট্যাগ ধরল এবং স্বয়ংক্রিয়ভাবে প্রোডাকশনের Git-এ কমিট করল। নতুন image-এ একটি ভাঙা migration ছিল; প্রোডাকশন অ্যাপ চালু হতে ব্যর্থ হলো। সকাল ৭টায় ব্যবহারকারীরা অভিযোগ করলেন।

টাইমলাইন: ০২:০৩ নতুন image প্রকাশ; ০২:০৫ Flux Git আপডেট; ০২:০৬ প্রোডাকশনে rollout; ০২:০৭ Pod CrashLoopBackOff; ০৭:১৫ অভিযোগ; ০৭:৪০ হাতে আগের tag-এ ফেরা।

প্রশ্ন: (১) স্বয়ংক্রিয় image আপডেটের নকশায় কী কী গার্ডরেল ছিল না? (২) CrashLoopBackOff রাতেই ধরা পড়ল না কেন — alert কোথায় ছিল? (৩) Rollback-এর সঠিক পদ্ধতি কী ছিল — Git revert, নাকি হাতে tag বদল? (৪) ভবিষ্যতে image promotion-এর নিরাপদ প্রবাহ কেমন হবে?

3. etcd বিভ্রাটে control plane অচল

ঘটনা: TechNova-র তিন সদস্যের etcd ক্লাস্টারের দুটি সদস্য একই সময়ে ডিস্ক সমস্যায় পড়ল। API server লেখা বন্ধ করে দিল; নতুন ডেপ্লয়, স্কেলিং ও Pod তৈরি সব বন্ধ। চলমান Pod-গুলো চলতে থাকল, কিন্তু কেউ কিছু বদলাতে পারছিল না।

টাইমলাইন: ১৪:০০ প্রথম etcd সদস্যের ডিস্ক সতর্কতা; ১৪:২০ দ্বিতীয় সদস্য ব্যর্থ; ১৪:২১ quorum হারাল; ১৪:৩০ API লেখা বন্ধ; ১৬:০০ snapshot থেকে এক সদস্য পুনরুদ্ধার; ১৭:৩০ quorum ফিরল।

প্রশ্ন: (১) দুটি সদস্য একসাথে ব্যর্থ হওয়ার সম্ভাব্য সাধারণ কারণ কী? (২) Quorum হারালে চলমান Pod কেন চলতে থাকে — control plane ও data plane-র পার্থক্য ব্যাখ্যা করুন। (৩) Snapshot থেকে পুনরুদ্ধারের ঝুঁকি কী — কোন ডেটা হারাতে পারে? (৪) ভবিষ্যতে etcd সদস্যদের আলাদা ব্যর্থতা-ক্ষেত্রে (failure domain) রাখার পরিকল্পনা লিখুন।

4. Service mesh আপগ্রেডে ট্রাফিক বিচ্ছিন্ন

ঘটনা: Istio control plane আপগ্রেডের সময় data plane proxy-গুলোর কনফিগারেশন আপডেট বন্ধ হয়ে গেল। নতুন Pod-গুলোতে পুরোনো কনফিগারেশন গেল, আর কিছু সার্ভিসের মধ্যে mTLS হাত মেলানো (handshake) ব্যর্থ হতে শুরু করল।

টাইমলাইন: ১১:০০ Istiod আপগ্রেড শুরু; ১১:১৫ নতুন Pod-এ sidecar পুরোনো কনফিগারেশন পেল; ১১:৩০ আন্তঃসার্ভিস ত্রুটি বাড়ল; ১২:০০ আপগ্রেড থামিয়ে আগের সংস্করণে ফেরা হলো।

প্রশ্ন: (১) Control plane ব্যর্থ হলে data plane কেন কিছুক্ষণ চলে — আর কখন থামে? (২) আপগ্রেডের আগে canary পদ্ধতিতে control plane পরীক্ষা কীভাবে করা যেত? (৩) mTLS handshake ব্যর্থতার নির্ণয়ে কোন তিনটি জিনিস পরীক্ষা করবেন? (৪) আপগ্রেড পরিকল্পনায় rollback-এর শর্ত কী লিখবেন?

5. ডেটাবেস মাইগ্রেশনে ডেপ্লয় ব্যর্থ

ঘটনা: একটি বড় ডেটাবেস মাইগ্রেশনসহ নতুন অ্যাপ সংস্করণ ডেপ্লয় করা হলো। মাইগ্রেশন অর্ধেক পথে ব্যর্থ হলো; অ্যাপের নতুন Pod-গুলো পুরোনো স্কিমায় চলতে পারল না, পুরোনো Pod-গুলোও নতুন স্কিমার অংশ দেখে ভাঙতে শুরু করল।

টাইমলাইন: ১৮:০০ ডেপ্লয় শুরু; ১৮:০৫ মাইগ্রেশন শুরু; ১৮:১২ মাইগ্রেশন ব্যর্থ; ১৮:১৫ নতুন ও পুরোনো — দুই সংস্করণই ত্রুটি দিচ্ছে; ১৯:৩০ ডেটাবেস ব্যাকআপ থেকে ফেরানো হলো।

প্রশ্ন: (১) Expand-and-contract প্যাটার্ন কীভাবে এই বিপর্যয় এড়াত? (২) মাইগ্রেশন ও অ্যাপ ডেপ্লয় একসাথে করা কেন ঝুঁকিপূর্ণ? (৩) Rollback-এর সময় ডেটা হারানোর ঝুঁকি কীভাবে মাপবেন? (৪) ভবিষ্যতে মাইগ্রেশনের জন্য কোন অনুমোদন ও পরীক্ষার ধাপ যোগ করবেন?

6. FinOps: নীরব খরচ বিস্ফোরণ

ঘটনা: TechNova-র মাসিক ক্লাউড বিল হঠাৎ ৪০% বেড়ে গেল। কোনো নতুন সার্ভিস চালু হয়নি। তদন্তে দেখা গেল একটি পরীক্ষামূলক (experiment) Namespace-এ বড় GPU নোড চলছে তিন সপ্তাহ ধরে, কেউ বন্ধ করেনি।

টাইমলাইন: মাসের শুরুতে পরীক্ষা চালু; ৩ সপ্তাহ পর বিলে ধরা পড়ল; কোনো বাজেট alert ছিল না।

প্রশ্ন: (১) খরচ তিন সপ্তাহ ধরে না ধরা পড়ার কারণ কী — দৃশ্যমানতার কোন ঘাটতি? (২) পরীক্ষামূলক পরিবেশে খরচ নিয়ন্ত্রণের তিনটি ব্যবস্থা লিখুন। (৩) Showback রিপোর্ট কীভাবে দায়িত্ব তৈরি করত? (৪) খরচের সীমা (budget alert) কোথায় বসাবেন — Namespace, দল, নাকি প্রকল্প স্তরে?

7. Zero-day দুর্বলতায় জরুরি প্যাচ

ঘটনা: একটি বহুল ব্যবহৃত বেস image-এ গুরুতর দুর্বলতা (CVE) প্রকাশ পেল। TechNova-র ৪০টি সার্ভিস একই বেস image ব্যবহার করে। প্যাচ করতে প্রতিটি image আবার তৈরি ও ডেপ্লয় করতে হবে।

টাইমলাইন: ০৯:০০ CVE প্রকাশ; ১০:০০ প্রভাবিত image-এর তালিকা তৈরি; ১২:০০ প্যাচ করা বেস image প্রস্তুত; ১৪:০০ প্রথম ৫টি সার্ভিস ডেপ্লয়; ২০:০০ বাকিগুলো বাকি।

প্রশ্ন: (১) প্রভাবিত সার্ভিসের তালিকা কীভাবে দ্রুত তৈরি করবেন — SBOM ও image inventory-র ভূমিকা কী? (২) ৪০টি সার্ভিসের মধ্যে কোনগুলো আগে প্যাচ করবেন — অগ্রাধিকার কীভাবে ঠিক করবেন? (৩) প্যাচের সময় সেবা ব্যাহত না করার কৌশল কী? (৪) ভবিষ্যতে বেস image আপডেট স্বয়ংক্রিয় করার পদ্ধতি লিখুন।

8. Multi-cluster নীতির অমিলে নিরাপত্তা ফাঁক

ঘটনা: নিরাপত্তা audit-এ দেখা গেল একটি ক্লাস্টারে NetworkPolicy ও Pod Security নীতি কঠোর, অন্য ক্লাস্টারে প্রায় নেই। একটি সংবেদনশীল অ্যাপ ভুল করে কম-নিরাপদ ক্লাস্টারে ডেপ্লয় হয়েছিল।

টাইমলাইন: নীতি হাতে প্রয়োগ করা হয়েছিল; GitOps-এ নীতির কোনো কেন্দ্রীয় উৎস ছিল না।

প্রশ্ন: (১) অমিলটি কীভাবে তৈরি হলো — প্রক্রিয়ার কোন ঘাটতি? (২) নীতিকে কোড হিসেবে কেন্দ্রীয় Git-এ রাখলে কী বদলাবে? (৩) নতুন ক্লাস্টার যোগ করার সময় নীতি যাচাইয়ের চেকলিস্ট লিখুন। (৪) ইতিমধ্যে ভুল ক্লাস্টারে চলা অ্যাপটির জন্য তাৎক্ষণিক পদক্ষেপ কী?

9. Observability ফাঁকা: নীরব ব্যর্থতা

ঘটনা: একটি পেমেন্ট সার্ভিস ২ ঘণ্টা ধরে কিছু অনুরোধ নীরবে ব্যর্থ করছিল, কিন্তু কোনো alert আসেনি। ড্যাশবোর্ড সবুজ ছিল, কারণ সাফল্যের হার সামগ্রিকভাবে বেশি দেখাচ্ছিল।

টাইমলাইন: ত্রুটি শুধু একটি নির্দিষ্ট পেমেন্ট পদ্ধতিতে; সামগ্রিক সাফল্যের হার 99% ছিল, কিন্তু ওই পদ্ধতিতে 40% ব্যর্থ।

প্রশ্ন: (১) সামগ্রিক মেট্রিক কীভাবে ব্যর্থতা লুকিয়ে ফেলল? (২) কোন ধরনের SLI এই ব্যর্থতা ধরত — ব্যাখ্যা করুন। (৩) Alert-এর নকশায় কী বদলাবেন? (৪) গ্রাহকের অভিযোগের আগে ধরার জন্য কোন পর্যবেক্ষণ যোগ করবেন?

10. Platform adoption-এ প্রতিরোধ

ঘটনা: প্ল্যাটফর্ম দল ছয় মাস ধরে একটি IDP বানাল, কিন্তু দলগুলো তা ব্যবহার করছে না। দলগুলো বলছে টেমপ্লেট তাদের কাজে লাগে না, ডকুমেন্টেশন পুরোনো, আর সাহায্য পেতে দেরি হয়।

টাইমলাইন: ৬ মাস উন্নয়ন, ব্যবহারকারী ছাড়া; চালুর পর ব্যবহার 10%-এর নিচে; দলগুলো নিজেদের স্ক্রিপ্ট ব্যবহার করছে।

প্রশ্ন: (১) ব্যর্থতার মূল কারণ কী — প্রযুক্তি, নাকি প্রক্রিয়া? (২) ব্যবহারকারীদের সাথে কথা বলে সমস্যা খুঁজে বের করার পরিকল্পনা লিখুন। (৩) পরের ৩০ দিনে কোন তিনটি ছোট উন্নতি করবেন? (৪) সাফল্য মাপার সূচক কী হবে — ব্যবহারকারীর সংখ্যা, নাকি সময় বাঁচা?

চূড়ান্ত পরীক্ষা • ১০০ নম্বর • ৩ ঘণ্টা

Final Examination — Practical (100 Marks, 3 Hours)

পরীক্ষার নির্দেশনা

- সময়: ৩ ঘণ্টা। মোট নম্বর: ১০০। পাস নম্বর: ৬০।
- Section A বাধ্যতামূলক। Section B থেকে যেকোনো ৩টি প্রশ্নের উত্তর দিন। Section C (নকশা) বাধ্যতামূলক।
- হ্যান্ডবুক অংশটি (Glossary ও Quick Reference) সাথে রাখা যাবে; ইন্টারনেট ব্যবহার করা যাবে না।
- নকশা প্রশ্নে চিত্র (ASCII/বর্ণনা) ও ট্রেড-অফ টেবিল দিলে বেশি নম্বর। শুধু টুলের নামের তালিকা দিলে নম্বর কমবে।

Section A — সংক্ষিপ্ত প্রশ্ন (২০ × ২ = ৪০ নম্বর)

1. Control plane-এর চারটি মূল উপাদানের নাম ও কাজ লিখুন।
2. etcd quorum হারালে কী হয় এবং কেন?
3. GitOps-এ drift বলতে কী বোঝায়?
4. Argo CD ও Flux-এর মধ্যে একটি মূল পার্থক্য লিখুন।
5. Helm values.yaml-এর উদ্দেশ্য কী?
6. Canary ও blue-green-এর মধ্যে পার্থক্য লিখুন।
7. Service mesh-এ data plane কী?
8. NetworkPolicy না থাকলে কী ঝুঁকি হয়?
9. StatefulSet কেন ডেটাবেসের জন্য উপযুক্ত?
10. HPA ও Cluster Autoscaler-এর দায়িত্ব আলাদা করুন।
11. 30 দিনে 99.9% SLO-র error budget কত? হিসাব দেখান। (ইঙ্গিত: 99.99% = 4 মিনিট 19 সেকেন্ড।)
12. Policy as Code-এর একটি উদাহরণ দিন।
13. Kubernetes Secret কেন এনক্রিপশন নয়?
14. FinOps-এ ট্যাগিং কেন প্রয়োজন?
15. RTO ও RPO-র পার্থক্য লিখুন।
16. OpenTelemetry-র সুবিধা কী?
17. Golden path বলতে কী বোঝায়?
18. Terraform state lock কেন দরকার?
19. Zero-trust-এর মূল নীতি লিখুন।
20. Backstage catalog-এ owner ক্ষেত্রের গুরুত্ব কী?

Section B — বর্ণনামূলক (৩ × ১০ = ৩০ নম্বর, যেকোনো ৩টি)

1. TechNova ঢাকা ও চট্টগ্রামে দুটি ক্লাস্টার চালাতে চায়। Multi-cluster নকশা করুন: GitOps, নীতি, পর্যবেক্ষণ ও failover — প্রতিটির সিদ্ধান্ত ও যুক্তি দিন।

2. একটি প্রোডাকশন ঘটনার বর্ণনা দেওয়া হলো: canary-তে ত্রুটি বাড়ছে, কিন্তু analysis gate সবুজ। আপনি কী করবেন — ধাপে ধাপে লিখুন, মেট্রিক বিলম্বের প্রসঙ্গসহ।
3. একটি দল service mesh নিতে চায়, আরেকটি দল মনে করে এটি অপ্রয়োজনীয়। দুই পক্ষের যুক্তি মূল্যায়ন করে আপনার সুপারিশ দিন — কখন mesh নেবেন, কখন নেবেন না।
4. TechNova-র মাসিক ক্লাউড বিল ৪০% বেড়েছে। FinOps তদন্ত ও প্রতিকার পরিকল্পনা লিখুন।
5. Terraform state নষ্ট হয়ে গেছে। পুনরুদ্ধারের ধাপ ও ভবিষ্যতে প্রতিরোধের ব্যবস্থা লিখুন।

Section C — স্থাপত্য নকশা (৩০ নম্বর, বাধ্যতামূলক)

TechNova (technova.example, 203.0.113.x) একটি নতুন প্ল্যাটফর্ম বানাতে চায়: দুটি ক্লাস্টার (ঢাকা ও চট্টগ্রাম), GitOps-ভিত্তিক ডেলিভারি, progressive delivery, কেন্দ্রীয় পর্যবেক্ষণ, নীতি-ভিত্তিক নিরাপত্তা ও FinOps নিয়ন্ত্রণ। আপনার উত্তরে থাকতে হবে: (ক) সামগ্রিক নকশা চিত্র ও উপাদান তালিকা (১০ নম্বর); (খ) প্রতিটি বড় সিদ্ধান্তের যুক্তি ও বাদ দেওয়া বিকল্প (১০ নম্বর); (গ) ব্যর্থতার পরিস্থিতি ও পুনরুদ্ধার পরিকল্পনা, SLO ও error budget-সহ (১০ নম্বর)।

Marking Scheme

Section	নম্বর	মূল্যায়নের দিক
A — সংক্ষিপ্ত	৪০	সঠিক ধারণা ও উদাহরণ (প্রতিটি ২)
B — বর্ণনামূলক	৩০	যুক্তি, ট্রেড-অফ, উদাহরণ (প্রতিটি ১০)
C — নকশা	৩০	নকশা ১০ + যুক্তি ১০ + ব্যর্থতা পরিকল্পনা ১০
মোট	১০০	পাস: ৬০; ভালো: ৭৫+; চমৎকার: ৮৫+

উত্তরপত্র • MCQ

Answer Key — 120 MCQs (সঠিক অক্ষর + সঠিক বিকল্পের পাঠ)

প্রতিটি প্রশ্নের সঠিক অক্ষর ও সঠিক বিকল্পের সম্পূর্ণ পাঠ দেওয়া হলো — অক্ষর ও পাঠ মিলিয়ে নিন, যাতে কোনো অমিল ধরা পড়ে। তারপর এক লাইনের ব্যাখ্যা পড়ুন।

উত্তর বণ্টন: A = 30, B = 30, C = 30, D = 30 (মোট ১২০)।

প্রশ্ন	সঠিক উত্তর	ব্যাখ্যা
1	**A** — etcd-র লেটেন্সি ও ডিস্ক I/O, কারণ API server-এর প্রতিটি লেখা etcd-তে যায়	API server stateless; ধীরগতির মূল কারণ প্রায়শই etcd-র ধীর ডিস্ক বা বেশি লেখা।
2	**B** — etcd-কে অন্তত তিনটি সদস্যের quorum-সহ আলাদা নোডে চালানো এবং API server-কে লোড ব্যালান্সারের পেছনে রাখা	তিন সদস্যের etcd quorum একটি নোড হারালেও কাজ করে; একক etcd single point of failure।
3	**C** — Pod-এর ভেতরে চলা অ্যাপ্লিকেশনের সোর্স কোডের গুণমান	Scheduler শুধু placement সিদ্ধান্ত নেয়; অ্যাপ কোডের গুণমান তার বিবেচ্য নয়।
4	**D** — Validating webhook ব্যর্থ হলে অনুরোধ প্রত্যাখ্যান করা (fail-closed), যদি নীতিটি নিরাপত্তা-সংবেদনশীল হয়	নিরাপত্তা-সংবেদনশীল নীতিতে fail-open হলে অননুমোদিত Pod ঢুকে পড়ে; fail-closed নিরাপদ।
5	**B** — Scheduler-র লগ ও নোডের উপলব্ধ resource — resource request মেটানোর মতো নোড আছে কি না	Pending মানে scheduling হয়নি; ImagePullBackOff ভিন্ন লক্ষণ। resource ঘাটতি সবচেয়ে সাধারণ কারণ।
6	**C** — নিয়মিত নেওয়া etcd snapshot থেকে পুনরুদ্ধার করা এবং snapshot-এর বয়স যাচাই করা	etcd-ই ক্লাস্টারের সত্যের উৎস; snapshot ছাড়া অবস্থা পুনরুদ্ধার করা যায় না।
7	**D** — কম লেটেন্সির SSD, কারণ etcd প্রতিটি লেখায় fsync করে	etcd-র লেখা fsync-নির্ভর; ধীর ডিস্ক সরাসরি API লেটেন্সি বাড়ায়।
8	**A** — 403 Forbidden — প্রমাণীকরণ সফল কিন্তু অনুমতি নেই	403 মানে পরিচয় ঠিক আছে, অনুমতি নেই; 401 মানে পরিচয় যাচাই হয়নি।
9	**C** — etcd শুধু পড়ার (read-only) অবস্থায় যায় বা লেখা বন্ধ করে দেয়, যাতে বিভক্ত মস্তিষ্ক (split-brain) না হয়	Quorum ছাড়া etcd লেখা বন্ধ করে দেয় — এটিই নিরাপত্তার জন্য নকশা করা আচরণ।
10	**D** — তৃতীয় পক্ষের API সার্ভারকে মূল API-তে যুক্ত করে নতুন রিসোর্স ধরন দেওয়া যায়, মূল API server বদল না করে	Aggregation layer এক্সটেনশন API যুক্ত করে; মূল API server অপরিবর্তিত থাকে।
11		

	A — আগে থেকে পরীক্ষা করা DNS/ Global Load Balancer failover, যাতে স্বাস্থ্য-পরীক্ষা ব্যর্থ হলে ট্রাফিক অন্য ক্লাস্টারে যায়	Kubernetes নিজে ক্লাস্টার-পার failover করে না; DNS বা Global LB দিয়ে আগে থেকে পরিকল্পনা করতে হয়।
12	**B** — একটি কেন্দ্রীয় Git রিপোজিটরি থেকে প্রতিটি ক্লাস্টারে GitOps দিয়ে একই নীতি ও কনফিগারেশন প্রয়োগ করা	GitOps কেন্দ্রীয় সত্যের উৎস দেয়; হাতে apply করলে ক্লাস্টারগুলোর মধ্যে drift তৈরি হয়।
13	**D** — একটি ক্লাস্টারের ব্যর্থতার বিস্ফোরণ-ব্যাসার্ধ (blast radius) অন্যটিতে ছড়ায় না	স্বাধীন ক্লাস্টার ব্যর্থতা আলাদা রাখে; federation জটিলতা ও ভাগ করা ব্যর্থতার ঝুঁকি আনে।
14	**A** — Ingress/Gateway দিয়ে প্রকাশিত HTTPS API দিয়ে যোগাযোগ, প্রয়োজনে mTLS যুক্ত করে	প্রকাশিত API দিয়ে শুরু করা সহজ ও নিরাপদ; ফ্ল্যাট নেটওয়ার্ক পরে, প্রয়োজনে বিবেচনা করা যায়।
15	**B** — প্রতিটি ক্লাস্টারের সংস্করণ, নীতি, পর্যবেক্ষণ ও আপগ্রেডের ভিন্নতা সামলানো	ভিন্নতা বাড়লেই drift, আপগ্রেড ঝুঁকি ও ডিবাগিং কঠিন হয়; মানকরণ (standardization) তাই মূল কাজ।
16	**C** — সিস্টেমের কাঙ্ক্ষিত অবস্থার একমাত্র সত্যের উৎস (single source of truth)	GitOps-এ Git-ই কাঙ্ক্ষিত অবস্থা নির্ধারণ করে; ক্লাস্টার সেই অবস্থার সাথে মিলিয়ে নেয়।
17	**A** — Git-এ কেউ নতুন কমিট করেছে, অথবা Argo CD-র sync সেটিংসে automated sync বন্ধ আছে	OutOfSync মানে Git ও ক্লাস্টারের অবস্থা মিলছে না; নতুন কমিটই সবচেয়ে সাধারণ কারণ।
18	**C** — ক্লাস্টারে হাতে করা পরিবর্তন Argo CD স্বয়ংক্রিয়ভাবে Git-এর অবস্থায় ফিরিয়ে আনে	Self-heal drift সংশোধন করে; এটি GitOps-এর ঘোষিত অবস্থা বজায় রাখার মূল ব্যবস্থা।
19	**D** — Git রিপোজিটরিতে ভুল কনফিগারেশন গেলে তা স্বয়ংক্রিয়ভাবে প্রোডাকশনে ছড়িয়ে পড়ে — তাই Pull Request review, automated sync-এ প্রুন ও self-heal সতর্কভাবে, এবং প্রয়োজনে manual approval ধাপ রাখা	GitOps ভুলকেও দ্রুত ছড়ায়; তাই review, নীতি-পরীক্ষা ও ধাপে ধাপে rollout প্রতিকার।
20	**B** — একই অ্যাপ অনেক ক্লাস্টার বা পরিবেশে টেমপ্লেট থেকে তৈরি করতে হলে	ApplicationSet টেমপ্লেট দিয়ে অনেক Application তৈরি করে; একক অ্যাপে এটি অপ্রয়োজনীয় জটিলতা।
21	**B** — GitRepository সোর্স নির্ধারণ করে, Kustomization সেই সোর্স থেকে কী প্রয়োগ করতে হবে তা নির্ধারণ করে	Flux-এ সোর্স ও প্রয়োগ আলাদা রিসোর্স; এই বিভাজনই পুনর্ব্যবহারযোগ্যতা দেয়।
22	**D** — Git ও API server-এ অপ্রয়োজনীয় চাপ বাড়ে এবং ছোট ভুল দ্রুত ছড়িয়ে পড়ে	খুব ঘন reconciliation চাপ বাড়ায়; interval পরিবেশের ঝুঁকি অনুযায়ী বেছে নিতে হয়।
23	**A** — নতুন image স্বয়ংক্রিয়ভাবে প্রোডাকশনে গেলে ভাঙা রিলিজ ঢুকে পড়তে পারে — তাই image policy ও পরীক্ষার ধাপ জরুরি	স্বয়ংক্রিয় image আপডেট সুবিধাজনক, তবে পরীক্ষা ছাড়া প্রোডাকশনে দেওয়া ঝুঁকিপূর্ণ।

24	**C** — দলটি GitOps Toolkit-এর কম্পোজেবল, CLI-কেন্দ্রিক পদ্ধতি চায় এবং UI-র চেয়ে Kubernetes নেটিভ রিসোর্সকে অগ্রাধিকার দেয়	পছন্দটি দলের কাজের ধরন ও সরঞ্জামের সাথে মিলের উপর নির্ভর করে; কোনো টুলই সর্বজনীনভাবে শ্রেষ্ঠ নয়।
25	**A** — Flux controller-র লগ ও Kustomization-এর status/conditions — কোন সোর্স বা YAML-এ ত্রুটি আছে	Status ও condition-ই Flux-এ ত্রুটির প্রথম সূত্র; সরাসরি Pod দেখলে মূল কারণ মিলবে না।
26	**B** — একই chart-কে পরিবেশভেদে ভিন্ন মান দিয়ে পুনর্ব্যবহার করা	Values দিয়ে একই chart dev, staging ও prod-এ ভিন্নভাবে চালানো যায়।
27	**C** — মূল YAML না বদলে overlay দিয়ে পরিবেশভেদে প্যাচ, নাম ও রেপ্লিকা বদলানো যায়	Kustomize টেমপ্লেট নয়, প্যাচ-ভিত্তিক; মূল ফাইল অপরিবর্তিত থাকে।
28	**D** — Helm, কারণ তৃতীয় পক্ষের প্যাকেজ করা অ্যাপে values দিয়ে কনফিগার করা ও সংস্করণ ধরে রাখা সহজ	প্যাকেজ করা, পুনর্ব্যবহারযোগ্য অ্যাপে Helm-এর values ও release ব্যবস্থাপনা সুবিধাজনক।
29	**B** — helm rollback দিয়ে আগের পরিচিত ভালো revision-এ ফেরা এবং ব্যর্থতার কারণ লগে যাচাই করা	Rollback পরিচিত অবস্থায় ফেরায়; মুছে আবার বানানো ডেটা ও কনফিগারেশন হারাতে পারে।
30	**C** — Values ফাইল Git-এ যায় ও helm history-তে সংরক্ষিত হয় — সিক্রেট ফাঁস হয়ে যায়	সিক্রেটের জন্য Sealed Secrets, External Secrets বা ভল্ট ব্যবহার করতে হয়, values নয়।
31	**D** — Data plane (sidecar proxy) ট্রাফিক বহন করে; control plane (যেমন Istiod) proxy-দের কনফিগারেশন দেয়	এই বিভাজন বোঝা mesh ডিবাগিংয়ের ভিত্তি — সমস্যা কোন স্তরে তা চিহ্নিত করা যায়।
32	**A** — উভয় Pod-এ sidecar proxy আছে কি না এবং destination rule-এ mTLS মোড কী	mTLS ব্যর্থতার সাধারণ কারণ: একটি পক্ষে sidecar নেই, অথবা মোডের অমিল।
33	**C** — সম্ভবত নয় — mesh-এর পরিচালন-ভার সুবিধার চেয়ে বেশি হতে পারে; প্রথমে Ingress + mTLS-এর সহজ বিকল্প বিবেচনা করুন	প্রযুক্তি বেছে নেওয়ার সিদ্ধান্ত সুবিধা বনাম পরিচালন-ভারের হিসাবে হয়; mesh সবার জন্য নয়।
34	**D** — VirtualService — ওজন (weight) দিয়ে ট্রাফিক ভাগ করা হয়	VirtualService-এ weight দিয়ে নতুন সংস্করণে ধাপে ধাপে ট্রাফিক পাঠানো যায়।
35	**A** — অ্যাপ কোড না বদলে প্রতিটি সার্ভিসের লেটেন্সি, ত্রুটি ও ট্রাফিকের স্বয়ংক্রিয় telemetry পাওয়া যায়	Sidecar proxy স্বয়ংক্রিয়ভাবে মেট্রিক ও ট্রেস দেয় — এটিই mesh-এর বড় প্রণোদনা।
36	**B** — Gateway API ভূমিকা-ভিত্তিক (infrastructure, cluster operator, developer) ও এক্সটেনসিবল — Ingress-এর annotation-নির্ভর সীমা কাটায়	Gateway API-র নকশা ভূমিকা আলাদা করে ও স্ট্যান্ডার্ড রিসোর্স দেয়; Ingress annotation-এ ভর করে।
37	**D** — ক্লাস্টারের নেটওয়ার্ক নীতি (NetworkPolicy), এনক্রিপশন ও পারফরম্যান্সের প্রয়োজন কী	

		CNI পছন্দ নীতি, নিরাপত্তা ও পারফরম্যান্সের প্রয়োজন দিয়ে ঠিক হয়, জনপ্রিয়তা দিয়ে নয়।
38	**A** — Policy-র podSelector ও ingress/egress নিয়ম — অনুমোদিত উৎস ও পোর্ট ঠিক আছে কি না	NetworkPolicy ডিফল্টভাবে অনুমতি দেয় না; ভুল selector বা পোর্টই সাধারণ কারণ।
39	**B** — TLS সমাপ্তি (termination) ও প্রমাণীকরণ Ingress/Gateway স্তরে কেন্দ্রীয়ভাবে করা	কেন্দ্রীয় TLS ও প্রমাণীকরণ ভুলের সুযোগ কমায়; সরাসরি প্রকাশ বিপজ্জনক।
40	**C** — <service>.<namespace>.svc.cluster.local — সম্পূর্ণ DNS নাম দিয়ে	একই Namespace-এ ছোট নাম চলে; ভিন্ন Namespace-এ সম্পূর্ণ DNS নাম লাগে।
41	**A** — Deployment Pod-দের স্থিতিশীল নেটওয়ার্ক পরিচয় ও স্থিতিশীল স্টোরেজ দেয় না; Pod বদলালে ডেটা ও পরিচয় হারাতে পারে	StatefulSet স্থিতিশীল নাম, ক্রমিক চালু ও volumeClaimTemplate দেয় — ডেটাবেসের জন্য এটিই প্রয়োজন।
42	**C** — স্টোরেজ সিস্টেমকে Kubernetes-এর সাথে যুক্ত করা — volume তৈরি, সংযুক্তি ও snapshot পরিচালনা	CSI ছাড়া প্রতিটি স্টোরেজের জন্য Kubernetes-এর ভেতরে কোড লিখতে হতো; CSI তা প্লাগইন করে।
43	**D** — Pod-এর সাথে বাঁধা ephemeral volume-কে স্থায়ী ডেটার জন্য ব্যবহার করা	Ephemeral volume Pod মুছলেই যায়; স্থায়ী ডেটার জন্য PersistentVolume দরকার।
44	**B** — PVC-র access mode, StorageClass ও reclaim policy — volume Pod-এর জীবনচক্রের বাইরে টিকে কি না	ডেটার স্থায়িত্ব volume-র জীবনচক্র দিয়ে নির্ধারিত হয়, Pod-এর নাম দিয়ে নয়।
45	**B** — Volume তখনই তৈরি হয় যখন Pod-এর নোড ঠিক হয় — ভুল availability zone-এ volume তৈরি হয় না	দেরিতে বাঁধা (late binding) zone-অমিলের সমস্যা এড়ায়; Immediate মোডে ভুল zone-এ volume তৈরি হতে পারে।
46	**D** — CPU/memory ব্যবহার বা কাস্টম মেট্রিক — লক্ষ্যমাত্রার তুলনায় বর্তমান ব্যবহার	HPA মেট্রিক দেখে replica সংখ্যা বদলায়; লগ বা সময় তার ইনপুট নয়।
47	**A** — Pod-এর resource request বেশি, অথবা image টানতে দেরি হয়, বা HPA-র মেট্রিক সংগ্রহে বিলম্ব	স্কেলিং দেরি প্রায়শই scheduling, image pull বা মেট্রিক বিলম্বের কারণে হয়।
48	**C** — HPA Pod বাড়ায়; Pod বসানোর জায়গা না থাকলে Cluster Autoscaler নোড বাড়ায় — দুটি একসাথে কাজ করে	HPA Pod স্তরে, Cluster Autoscaler নোড স্তরে কাজ করে; দুটি পরিপূরক।
49	**A** — নোডে বেশি Pod বসে, চাপ বাড়লে CPU থ্রটলিং বা OOMKill হয়	কম request overcommitment আনে; চাপে Pod-গুলো একে অপরের resource কেড়ে নেয়।
50	**B** — VPA সুপারিশ প্রয়োগ করতে Pod পুনরায় চালু করে — প্রোডাকশনে হঠাৎ restart ব্যাঘাত ঘটাতে পারে	VPA-র restart আচরণ জানা জরুরি; অনেক দল প্রথমে recommendation-only মোডে চালায়।

51	**C** — নতুন সংস্করণে অল্প ট্রাফিক পাঠিয়ে ঝুঁকি যাচাই করা, তারপর ধাপে ধাপে বাড়ানো	Canary ঝুঁকি সীমিত করে; সব ট্রাফিক একসাথে পাঠানো blue-green বা recreate-এর কাজ।
52	**D** — Rollout স্বয়ংক্রিয়ভাবে থেমে যায় বা আগের স্থিতিশীল সংস্করণে ফিরে যায়	Analysis gate-ই progressive delivery-র নিরাপত্তা; মেট্রিক খারাপ হলে এগোনো বন্ধ হয়।
53	**B** — Blue-green-এ পুরো নতুন পরিবেশ তৈরি করে একসাথে সুইচ করা হয়; canary-তে ট্রাফিক ধাপে ধাপে ভাগ করা হয়	Blue-green একসাথে সুইচ; canary ধাপে ধাপে — ঝুঁকি ও খরচের ভিন্নতা এখানেই।
54	**C** — নির্ভরযোগ্য মেট্রিক ও স্বাস্থ্য-পরীক্ষা, যাতে স্বয়ংক্রিয় সিদ্ধান্ত ভুল না হয়	স্বয়ংক্রিয় promotion মেট্রিকের উপর দাঁড়ায়; মেট্রিক ভুল হলে ভাঙা রিলিজও এগিয়ে যায়।
55	**D** — Analysis-এর সময়কাল ও মেট্রিকের বিলম্ব বিবেচনা করে ধাপের সময় বাড়ানো এবং সতর্কতা (alert) যুক্ত করা	মেট্রিক বিলম্ব থাকলে ছোট ধাপে দ্রুত promotion ভুল নিরাপত্তা-বোধ দেয়।
56	**A** — ডেভেলপারদের জন্য একটি কেন্দ্রীয় পোর্টাল — সার্ভিস ক্যাটালগ, টেমপ্লেট, ডকুমেন্টেশন ও সরঞ্জাম এক জায়গায়	Backstage IDP-র মুখ; এটি সার্ভিস খোঁজা ও নতুন সার্ভিস বানানো সহজ করে।
57	**C** — নতুন সার্ভিস সবসময় দলের মান অনুযায়ী — একই কাঠামো, CI, পরীক্ষা ও পর্যবেক্ষণসহ — তৈরি হয়	Golden path টেমপ্লেটে বাঁধা থাকলে প্রতিটি নতুন সার্ভিস মানসম্মত হয়।
58	**D** — ঘটনার সময় কাকে ডাকতে হবে, তা জানা যায় এবং দায়িত্ব অস্পষ্ট থাকে না	মালিকানা অস্পষ্ট থাকলে incident-এ সময় নষ্ট হয়; catalog দায়িত্ব স্পষ্ট করে।
59	**A** — ডেভেলপারদের না জিজ্ঞেস করে শুধু প্ল্যাটফর্ম দলের পছন্দে সরঞ্জাম বানানো	IDP ব্যবহারকারীর সমস্যা সমাধান করতে হয়; ব্যবহারকারী ছাড়া বানানো প্ল্যাটফর্ম কেউ ব্যবহার করে না।
60	**B** — Catalog তথ্য কোড থেকে (যেমন catalog-info.yaml) স্বয়ংক্রিয়ভাবে হালনাগাদ করা, হাতে সম্পাদনা নয়	কোড-থেকে-তথ্য (docs-as-code) পদ্ধতি catalog-কে স্বয়ংক্রিয়ভাবে সত্য রাখে।
61	**D** — দলের অনুমোদিত, পরীক্ষিত ও সমর্থিত একটি প্রমিত পথ — নতুন সার্ভিস বানানো থেকে ডেপ্লয় পর্যন্ত	Golden path সহজ ও নিরাপদ পথ দেয়, তবে প্রয়োজনে সরে যাওয়ার সুযোগও থাকা উচিত।
62	**A** — ডেভেলপারদের সময় বাঁচছে কি না ও নতুন সার্ভিস চালু করতে কত সময় লাগে	প্ল্যাটফর্মের মূল্য ব্যবহারকারীর সময় বাঁচানোয়; টুলের সংখ্যা সাফল্য নয়।
63	**B** — কেন মানছে না, তা শোনা — path-টি তাদের সমস্যা সমাধান করছে কি না, তা বোঝা	অব্যবহৃত প্ল্যাটফর্মের কারণ প্রায়শই পথের ত্রুটি; শোনা ছাড়া ঠিক করা যায় না।
64	**C** — প্ল্যাটফর্মকে পণ্যের মতো ভাবা — ব্যবহারকারীর প্রয়োজন, ফিডব্যাক ও উন্নতির চক্র নিয়ে	পণ্য-মানসিকতা মানে ব্যবহারকারীর সমস্যা শুনে ধাপে ধাপে উন্নতি করা।
65	**A** — টুলটি আসল সমস্যা সমাধান করে কি না, ব্যবহার করা কতটা কঠিন, এবং বিকল্প কী আছে	অব্যবহারের কারণ না জেনে প্রচার বা মুছে ফেলা — দুটোই ভুল হতে পারে।

66	**C** — ছোট, ঘন ঘন মার্জ — দীর্ঘ শাখার মার্জ-সংঘাত ও পুরোনো কোডের ঝুঁকি কমে	ঘন ঘন ছোট মার্জ integration ঝুঁকি কমায়; দীর্ঘ শাখা ঝুঁকি বাড়ায়।
67	**D** — ব্যর্থতা দ্রুত জানানো এবং মূল শাখা (main) সবসময় সবুজ রাখা — ভাঙা main-এ নতুন কাজ না তোলা	সবুজ main ডেপ্লয়ের ভিত্তি; ভাঙা main-এ কাজ তুললে সবাই আটকে যায়।
68	**B** — উচ্চ-ঝুঁকির পরিবর্তনে — যেমন ডেটাবেস মাইগ্রেশন বা নিয়ন্ত্রিত পরিবেশে — মানুষের পর্যালোচনা যোগ করলে	Approval ঝুঁকি অনুযায়ী ব্যবহার করতে হয়; সব জায়গায় approval গতি মেরে ফেলে।
69	**B** — পুরোনো ক্যাশ থেকে ভুল নির্ভরতা (dependency) ব্যবহার করে ভুলভাবে পাস করা	Stale cache ভুল নিরাপত্তা-বোধ দেয়; ক্যাশের চাবি (key) সঠিকভাবে বাঁধতে হয়।
70	**D** — কোন অংশ বদলেছে তা ধরে শুধু প্রভাবিত অংশের পরীক্ষা চালানো (affected testing)	মনোরেপোতে সব পরীক্ষা সবসময় চালানো ধীর; affected analysis-ই স্কেলের চাবি।
71	**A** — দলের সবাই একই state দেখতে পায় এবং state lock দিয়ে একসাথে পরিবর্তনের সংঘাত এড়ানো যায়	Remote state + lock ছাড়া দলগত Terraform-এ state নষ্ট হওয়ার ঝুঁকি থাকে।
72	**C** — Module-র নতুন সংস্করণ হঠাৎ আচরণ বদলে দিলে প্রোডাকশন ভাঙতে পারে — পিন করা পরিবর্তন নিয়ন্ত্রণ করে	অপিন করা নির্ভরতা নীরবে বদলে যায়; পিন করা ইচ্ছাকৃত আপগ্রেড নিশ্চিত করে।
73	**A** — Apply বন্ধ করে কেন ধ্বংস দেখাচ্ছে তা খুঁজে বের করা — নাম, count বা চলকের পরিবর্তন কি না	অপ্রত্যাশিত destroy ডেটা হারানোর সংকেত; কারণ না জেনে Apply করা বিপজ্জনক।
74	**B** — State-কে এনক্রিপ্টেড remote backend-এ রাখা, সিক্রেটের জন্য ভল্ট ব্যবহার করা এবং state-এ সংবেদনশীল মান এড়ানো	State-এ সিক্রেট থাকতে পারে; তাই backend এনক্রিপশন ও কঠোর অ্যাক্সেস নিয়ন্ত্রণ জরুরি।
75	**C** — পরিবেশগুলোর কাঠামো ভিন্ন হলে বা আলাদা অনুমতি ও পর্যালোচনা চাইলে	Workspace একই কোডে ভিন্ন state দেয়; কাঠামো ভিন্ন হলে ডিরেক্টরি আলাদা করা স্পষ্টতর।
76	**D** — একই playbook বারবার চালালেও দ্বিতীয়বার আর পরিবর্তন হয় না — নিরাপদে পুনরায় চালানো যায়	Idempotent playbook ভুল করে দুবার চালালেও বিপদ হয় না।
77	**B** — কোন হোস্টে কী চালাতে হবে এবং হোস্টগুলো কীভাবে গোষ্ঠীবদ্ধ, তা নির্ধারণ করা	Inventory ছাড়া Ansible জানে না কোথায় কাজ করতে হবে।
78	**C** — Fork বাড়ানো, কৌশল (strategy) যাচাই করা এবং gathering facts সীমিত করা	Fork ও fact gathering-ই বড় পরিবেশে গতির প্রধান নিয়ামক।
79	**D** — সিক্রেট এনক্রিপ্ট করে Git-এ নিরাপদে রাখা	Vault ছাড়া সিক্রেট প্লেইন টেক্সটে Git-এ গেলে ফাঁস হয়।
80	**A** — দুটি কাজের জন্য আলাদা টুলের শক্তি বিবেচনা করা — Terraform provisioning-এ, Ansible কনফিগারেশনে শক্তিশালী; এক টুলে জোর করে দুটো করা প্রায়শই ভুল	প্রতিটি টুলের শক্তির জায়গা আলাদা; সঠিক বিভাজনই টেকসই নকশা।

81	**C** — Monitoring জানা প্রশ্নের উত্তর দেয়; observability অজানা প্রশ্নের উত্তর দেওয়ার ক্ষমতা দেয় — লগ, মেট্রিক ও ট্রেস একসাথে	Observability মানে সিস্টেমের ভেতরের অবস্থা বাইরে থেকে অনুসন্ধান করে বোঝা।
82	**D** — একাধিক span — প্রতিটি সার্ভিসে একটি কাজের অংশ, parent-child সম্পর্কে বাঁধা	Span-গুলো মিলে অনুরোধের পুরো পথ দেখায়; একটিমাত্র লগ তা পারে না।
83	**A** — ভেন্ডর-নিরপেক্ষ (vendor-neutral) instrumentation — একটি SDK দিয়ে ডেটা যেকোনো backend-এ পাঠানো যায়	Vendor-neutral হওয়ায় backend বদলালেও instrumentation বদলাতে হয় না।
84	**B** — Trace দেখে প্রতিটি span-এর সময় বিশ্লেষণ করা — কোন ধাপে সময় যাচ্ছে	Trace-ই অনুরোধের পথ ধরে ধীর ধাপ চিহ্নিত করে; অনুমানে resource বাড়ানো অপচয়।
85	**D** — প্রতিটি নতুন মান নতুন time series তৈরি করে — স্টোরেজ ও খরচ বিস্ফোরিত হয়	High-cardinality লেবেল মেট্রিক স্টোরেজের সবচেয়ে সাধারণ খরচ-বিস্ফোরণের কারণ।
86	**A** — 4 মিনিট 19 সেকেন্ড	30 দিনে 0.01% হলো 4m 19s — এই হিসাবটি মুখস্থ নয়, যাচাই করে নিতে হয়।
87	**B** — নতুন ফিচার ডেপ্লয় থামিয়ে নির্ভরযোগ্যতার কাজে মনোযোগ দেওয়া	Error budget-এর উদ্দেশ্যই হলো গতি বনাম নির্ভরযোগ্যতার ভারসাম্য বজায় রাখা।
88	**C** — SLI পরিমাপ, SLO লক্ষ্য, SLA চুক্তিগত প্রতিশ্রুতি (প্রায়শই জরিমানাসহ)	পরিমাপ, লক্ষ্য ও চুক্তি — তিনটি ভিন্ন স্তর; গুলিয়ে ফেললে সিদ্ধান্ত ভুল হয়।
89	**A** — Error budget কত দ্রুত শেষ হচ্ছে তা দেখে সমস্যা বড় হওয়ার আগেই সতর্ক করা যায়	Burn rate দ্রুত-পোড়া বনাম ধীর-পোড়া আলাদা করে; সাধারণ threshold alert তা পারে না।
90	**C** — ব্যবহারকারীর আসল প্রয়োজন যাচাই করে SLO পুনর্বিবেচনা করা — প্রতিটি অতিরিক্ত 9-এর খরচ ও সুবিধার হিসাব করা	প্রতিটি 9-এর খরচ বাড়ে; SLO ব্যবসায়িক প্রয়োজন দিয়ে ঠিক হয়, সর্বোচ্চ সংখ্যা দিয়ে নয়।
91	**D** — Admission স্তরে নীতি প্রয়োগ করা — নীতি-বিরুদ্ধ রিসোর্স ক্লাস্টারে ঢুকতে দেওয়া না	Policy as Code admission-এ বসে; ভুল রিসোর্স আগেই আটকানো যায়।
92	**B** — দলের ভাষা-পছন্দ ও নীতির জটিলতা — Kyverno YAML-ভিত্তিক, OPA Rego-ভিত্তিক ও বেশি নমনীয়	ভাষা ও জটিলতার মিলই দীর্ঘমেয়াদে নীতি রক্ষণাবেক্ষণ নির্ধারণ করে।
93	**B** — Image স্বাক্ষর যাচাই (signing/verification) ও নির্ভরযোগ্য রেজিস্ট্রি থেকে শুধু অনুমোদিত image ব্যবহার করা	স্বাক্ষর যাচাই ছাড়া যেকোনো image-ই আক্রমণের পথ হতে পারে।
94	**D** — Pod Security Standards / নীতি দিয়ে runAsNonRoot ও readOnlyRootFilesystem বাধ্যতামূলক করা	নীতি দিয়ে প্রতিরোধ করলে ভবিষ্যতের Pod-ও নিরাপদ থাকে; শুধু মুছে দিলে সমস্যা ফিরে আসে।
95	**A** — নীতির ঝুঁকি দিয়ে — নিরাপত্তা-সংবেদনশীল নীতিতে fail-closed, কম ঝুঁকির নীতিতে সতর্কতার সাথে fail-open বিবেচনা	Fail-closed নিরাপদ কিন্তু ডেপ্লয় থামাতে পারে; ঝুঁকি অনুযায়ী সিদ্ধান্ত নিতে হয়।

96	**C** — ক্লাউড খরচের দৃশ্যমানতা, দায়িত্ব ও নিয়ন্ত্রণ — খরচকে প্রকৌশল সিদ্ধান্তের অংশ করা	FinOps মানে অন্ধভাবে কাটছাঁট নয়; মানে খরচ বুঝে সিদ্ধান্ত নেওয়া।
97	**A** — Label ও ট্যাগ দিয়ে খরচ বরাদ্দ (allocation) করা — প্রতিটি Namespace, দল ও সার্ভিসের খরচ আলাদা দেখা	বরাদ্দ ছাড়া খরচের দায়িত্ব নির্ধারণ করা যায় না; ট্যাগিং-ই ভিত্তি।
98	**B** — Request করা resource-এর তুলনায় প্রকৃত ব্যবহার ধারাবাহিকভাবে অনেক কম	ব্যবহার বনাম request-এর ব্যবধানই অপচয়ের পরিমাপ।
99	**C** — যেকোনো সময় instance ফিরিয়ে নেওয়া (preemption) হতে পারে — তাই শুধু সহনশীল (fault-tolerant) কাজে ব্যবহার করা উচিত	Spot সস্তা, তবে বাধা (interruption) সামলানোর নকশা ছাড়া ব্যবহার ঝুঁকিপূর্ণ।
100	**D** — Showback খরচ দেখায়, কিন্তু টাকা কাটে না; chargeback আসলেই দলের বাজেট থেকে খরচ কাটে	Showback দিয়ে শুরু করে দায়িত্বের সংস্কৃতি গড়া যায়; chargeback আরও কঠোর পদক্ষেপ।
101	**B** — RTO কতক্ষণে ফিরে আসতে হবে; RPO কতটুকু ডেটা হারানো সহ্য করা যায়	RTO সময়ের লক্ষ্য, RPO ডেটা-হারানোর লক্ষ্য — দুটি একসাথে DR পরিকল্পনা নির্ধারণ করে।
102	**C** — নিয়মিত মহড়া (drill) দিয়ে পরীক্ষা করে — কাগজে লেখা পরিকল্পনা যথেষ্ট নয়	অপরীক্ষিত DR পরিকল্পনা প্রায়শই ব্যর্থ হয়; মহড়াই প্রমাণ।
103	**D** — নিয়ন্ত্রিতভাবে ব্যর্থতা ঘটিয়ে সিস্টেমের দুর্বলতা আগেই খুঁজে বের করা	Chaos পরীক্ষা প্রোডাকশনের অপ্রত্যাশিত ব্যর্থতার আগেই দুর্বলতা দেখায়।
104	**A** — ব্যাকআপটি আসলে কাজ নাও করতে পারে — দূষিত, অসম্পূর্ণ বা ভুল পদ্ধতির হতে পারে	অপরীক্ষিত ব্যাকআপ সবচেয়ে বিপজ্জনক অনুমান; পুনরুদ্ধার পরীক্ষাই প্রমাণ।
105	**C** — Blast radius সীমিত রাখা ও দ্রুত থামানোর (abort) ব্যবস্থা — প্রয়োজনে পরীক্ষা বন্ধ করা যায়	নিয়ন্ত্রিত পরীক্ষায়ও থামানোর পথ থাকতে হয়; নইলে পরীক্ষাই বিপর্যয় হয়।
106	**D** — প্রতিটি ক্লাউডের আলাদা API, পরিচয়, নেটওয়ার্ক ও খরচের মডেল সামলানো	ভিন্নতা সামলানোর খরচই multi-cloud-এর মূল্য; বিমূর্তকরণ (abstraction) ছাড়া তা বাড়ে।
107	**A** — অন-প্রিমিসিস ও ক্লাউডের মধ্যে নেটওয়ার্ক সংযোগ — লেটেন্সি, নিরাপত্তা ও নির্ভরযোগ্যতা	Hybrid-এর সেতুই নেটওয়ার্ক; সেটি ধীর বা অনিরাপদ হলে পুরো নকশা ভেঙে যায়।
108	**B** — খোলা মান (open standards) ও পোর্টেবল সরঞ্জাম — Kubernetes, Terraform, OpenTelemetry — ব্যবহার করা	পোর্টেবিলিটি মানে যেকোনো মুহূর্তে সরে যাওয়া নয়; মানে সরে যাওয়ার ক্ষমতা রাখা।
109	**D** — Managed service-এর গতি ও সুবিধা বনাম lock-in ও খরচ — কোনটি ব্যবসার জন্য গুরুত্বপূর্ণ, তা মাপা	Lock-in বনাম সুবিধার হিসাব ব্যবসায়িক; কোনো সর্বজনীন উত্তর নেই।
110	**A** — কেন্দ্রীয় পরিচয় প্রদানকারী (IdP) থেকে federation — প্রতিটি ক্লাউডে আলাদা ব্যবহারকারী না বানানো	Federation এক জায়গায় পরিচয় নিয়ন্ত্রণ করে; আলাদা অ্যাকাউন্ট ছড়িয়ে গেলে নিয়ন্ত্রণ হারায়।
111		

	B — কাউকে স্বয়ংক্রিয়ভাবে বিশ্বাস নয় — প্রতিটি অনুরোধ যাচাই করা, নেটওয়ার্কের ভেতরেও	Zero-trust মানে অবস্থান দিয়ে বিশ্বাস নয়; পরিচয় ও প্রসঙ্গ দিয়ে যাচাই।
112	**C** — Secret base64-এ থাকে, এনক্রিপ্টেড নয় — যে কেউ ডিকোড করতে পারে	Base64 এনকোডিং, এনক্রিপশন নয়; Git-এ সিক্রেট রাখতে Sealed Secrets বা External Secrets লাগে।
113	**A** — ভল্টের মতো বাইরের উৎস থেকে সিক্রেট Kubernetes Secret-এ সিঙ্ক করা — সিক্রেট Git-এ থাকে না	বাইরের ভল্টই সত্যের উৎস থাকে; Kubernetes শুধু ব্যবহারের সময় সিক্রেট পায়।
114	**C** — Pod-কে ক্লাউড পরিচয় দেওয়া যায়, দীর্ঘমেয়াদি চাবি (key) ছাড়া — চাবি ফাঁসের ঝুঁকি কমে	স্বল্পমেয়াদি পরিচয় চাবির চেয়ে নিরাপদ; চাবি ঘোরানোর (rotation) ভারও কমে।
115	**D** — সিক্রেটটি অবিলম্বে বাতিল (revoke) ও ঘোরানো (rotate) এবং কোথায় ব্যবহৃত হয়েছে তা খুঁজে বের করা	ফাঁসের পর প্রথম কাজ ক্ষতি সীমিত করা — revoke ও rotate; তদন্ত তারপর।
116	**B** — শুধু টুল বদলানো, কিন্তু প্রক্রিয়া, দায়িত্ব ও সংস্কৃতি না বদলানো	টুল ছাড়া সংস্কৃতি বদলায় না; transformation মানুষ ও প্রক্রিয়ার পরিবর্তন।
117	**B** — স্বেচ্ছায় ব্যবহারকারীর সংখ্যা ও তাদের সময় বাঁচা — জোর করে ব্যবহার নয়	স্বেচ্ছায় ব্যবহারই মূল্যের প্রমাণ; জোর করে ব্যবহার সংখ্যা বাড়ায়, মূল্য নয়।
118	**D** — একটি ছোট দলে পাইলট করে শেখা, তারপর প্রমাণসহ ছড়িয়ে দেওয়া	পাইলট ঝুঁকি সীমিত করে ও প্রমাণ তৈরি করে; big-bang পরিবর্তন প্রায়শই ব্যর্থ হয়।
119	**A** — সিদ্ধান্তের যুক্তি রক্ষা করা — কেন এই নকশা, কী বাদ দেওয়া হলো, ঝুঁকি কী — শুধু কাজ করলেই নয়	স্থপতির কাজ সিদ্ধান্ত ও তার যুক্তি; টুলের সংখ্যা গভীরতা প্রমাণ করে না।
120	**C** — প্রতিটি স্প্রিন্টে debt-এর জন্য নির্দিষ্ট সময় রাখা এবং debt-কে দৃশ্যমান (backlog-এ) করা	Debt দৃশ্যমান ও নিয়মিত শোধ না করলে তা নীরবে বাড়ে এবং গতি থামিয়ে দেয়।

উত্তরপত্র • ব্যাখ্যা ও রুব্রিক

Answer Guide-2 — Short Answers, Scenarios, Case Studies, Final Exam ও Chapter Mapping

২.১ Short Answer — মডেল উত্তর (সংক্ষিপ্ত)

1. **Control plane ব্যর্থ হলে চলমান Pod:** চলমান Pod চলতে থাকে, কারণ kubelet ও data plane স্বাধীনভাবে কাজ করে; তবে নতুন Pod, স্কেলিং ও পরিবর্তন বন্ধ হয়ে যায়।
2. **etcd snapshot:** নিয়মিত (যেমন দৈনিক) snapshot নিন, নিরাপদ জায়গায় রাখুন, এবং পুনরুদ্ধার মহড়া দিয়ে যাচাই করুন — অপরীক্ষিত snapshot-এর কোনো মূল্য নেই।
3. **Blast radius সীমিত করা:** স্বাধীন ক্লাস্টার, Namespace-ভিত্তিক quota ও নীতি, এবং ধাপে ধাপে rollout — এক জায়গার ব্যর্থতা অন্য জায়গায় ছড়ায় না।
4. **Drift:** বাস্তব অবস্থা Git-এর কাঙ্ক্ষিত অবস্থা থেকে সরে যাওয়া; Argo CD diff দেখিয়ে OutOfSync জানায়, self-heal থাকলে সংশোধন করে।
5. **Flux reconciliation:** Flux নিয়মিত Git পড়ে, কাঙ্ক্ষিত অবস্থার সাথে ক্লাস্টার মেলায়; interval অনুযায়ী এই চক্র চলে।
6. **Helm বনাম Kustomize:** প্যাকেজ করা তৃতীয় পক্ষের অ্যাপে Helm; নিজের YAML-এ পরিবেশভেদে প্যাচ দিতে Kustomize।
7. **Mesh ছাড়া mTLS:** Ingress-এ TLS, অ্যাপ-স্তরে TLS, অথবা CNI-ভিত্তিক এনক্রিপশন — তবে সার্ভিস-স্তরের নীতি ও telemetry mesh-এর মতো পাওয়া যায় না।
8. **Gateway API:** ভূমিকা আলাদা করে (infra/cluster/dev), স্ট্যান্ডার্ড রিসোর্স দেয়; Ingress annotation-নির্ভর ও প্রসারণে সীমিত।
9. **StatefulSet:** স্থিতিশীল নাম, স্থিতিশীল স্টোরেজ, ক্রমিক চালু — ডেটাবেসের পরিচয় ও ডেটা রক্ষা করে।
10. **HPA/VPA/Cluster Autoscaler:** HPA Pod-এর সংখ্যা, VPA Pod-এর আকার, Cluster Autoscaler নোডের সংখ্যা বদলায়।
11. **Canary বনাম blue-green:** Canary ধাপে ধাপে, কম ঝুঁকি, বেশি সময়; blue-green একসাথে সুইচ, বেশি resource, দ্রুত ফিরে যাওয়া।
12. **Catalog owner:** ঘটনার সময় দায়িত্বশীল দলকে দ্রুত ডাকা যায়; মালিকানা অস্পষ্ট থাকলে সময় নষ্ট হয়।
13. **Golden path বনাম paved road:** দুটোই প্রমিত পথ; paved road-এ সরে যাওয়ার সুযোগ স্পষ্টভাবে থাকে, golden path কখনো কঠোরভাবে বোঝানো হয় — মূল কথা এক: সহজ পথটি নিরাপদ পথ।
14. **দীর্ঘ শাখার ঝুঁকি:** মার্জ-সংঘাত, পুরোনো কোড, integration-এ দেরিতে ধরা পড়া ভুল।
15. **State lock:** একসাথে দুজন Apply করলে state নষ্ট হয়; lock একসাথে একজনকে Apply করতে দেয়।
16. **Idempotent Ansible উদাহরণ:** `apt: name=nginx state=present` — দ্বিতীয়বার চালালে আর কিছু করে না।
17. **Trace/span/log:** Trace একটি অনুরোধের পুরো পথ; span তার একটি ধাপ; log একটি ঘটনার রেকর্ড।
18. **99.9% বনাম 99.99%:** 30 দিনে 99.9% = 43 মিনিট 12 সেকেন্ড; 99.99% = 4 মিনিট 19 সেকেন্ড — দশগুণ পার্থক্য।
19. **Policy as Code:** নীতি কোডে লেখা — যেমন “সব Pod-এ runAsNonRoot বাধ্যতামূলক” — Git-এ পর্যালোচনা হয়, admission-এ প্রয়োগ হয়।

20. **ট্যাগিং না থাকলে:** কোন দল বা সার্ভিস কত খরচ করছে, তা জানা যায় না; দায়িত্ব নির্ধারণ অসম্ভব।
21. **RTO/RPO উদাহরণ:** “RTO 4 ঘণ্টা, RPO 1 ঘণ্টা” মানে 4 ঘণ্টায় সেবা ফিরবে, সর্বোচ্চ 1 ঘণ্টার ডেটা হারাতে পারে।
22. **Abstraction layer:** সুবিধা — পোর্টেবিলিটি; খরচ — অতিরিক্ত স্তর, সীমিত বিশেষ সুবিধা, পরিচালন-ভার।
23. **Secret এনক্রিপশন নয়:** Secret শুধু base64-এ থাকে; নিরাপদ বিকল্প — Sealed Secrets, External Secrets, ভল্ট।
24. **পাইলট দল বেছে নেওয়া:** পরিবর্তনে আগ্রহী, বাস্তব সমস্যা আছে, ছোট ও সহযোগী দল — সাফল্যের গল্প তৈরি করতে পারে এমন দল।
25. **App of Apps:** অনেক অ্যাপ একসাথে পরিচালনা করতে — একটি মূল Application থেকে বাকিগুলো তৈরি হয়।
26. **ভুল মেট্রিকের ঝুঁকি:** ভাঙা রিলিজও analysis পাস করে এগিয়ে যায়; মেট্রিকই স্বয়ংক্রিয় সিদ্ধান্তের ভিত্তি।
27. **Collector-এর সুবিধা:** কেন্দ্রীয়ভাবে ফিল্টার, ব্যাচ ও রাউট করা যায়; প্রতিটি অ্যাপকে backend জানতে হয় না।
28. **Chaos blast radius:** একটি Namespace-এ সীমাবদ্ধ করা, শুধু non-critical সার্ভিস লক্ষ্য করা, abort ব্যবস্থা রাখা।
29. **IDP সাফল্যের সূচক:** নতুন সার্ভিস চালু করতে সময়, স্বেচ্ছায় ব্যবহারকারীর সংখ্যা, ডেভেলপার সন্তুষ্টি।
30. **Main ভেঙে গেলে:** নতুন মার্জ থামানো, ভাঙা পরিবর্তন revert করা, সবাইকে জানানো — main সবুজ রাখা প্রথম অগ্রাধিকার।

২.২ Scenario — মূল্যায়নের নির্দেশনা

প্রতিটি scenario-তে পূর্ণ নম্বর পেতে হলে: (ক) লক্ষণ থেকে মূল কারণের অনুমান লিখুন; (খ) যাচাইয়ের ক্রম দিন (কী দেখবেন, কেন); (গ) তাৎক্ষণিক প্রতিকার ও স্থায়ী প্রতিকার আলাদা করুন; (ঘ) ভবিষ্যতে প্রতিরোধের ব্যবস্থা লিখুন। শুধু “Pod পুনরায় চালু করুন” লিখলে অর্ধেকের কম নম্বর। মূল ইঙ্গিত: Argo CD দৃশ্যকল্পে অ্যাপ নিজে রিসোর্স বদলালে ignoreDifferences সাময়িক, নকশা বদল স্থায়ী সমাধান; etcd দৃশ্যকল্পে snapshot যাচাই ও compaction নীতি; canary দৃশ্যকল্পে মেট্রিক বিলম্ব বিবেচনা করে ধাপের সময় বাড়ানো; খরচ দৃশ্যকল্পে ট্যাগিং ও বাজেট alert; mTLS দৃশ্যকল্পে sidecar ইনজেকশন যাচাই।

২.৩ Case Study — মূল্যায়নের নির্দেশনা

প্রতিটি কেসে ব্যর্থতার শৃঙ্খল (chain) চিহ্নিত করাই মূল কাজ — একটি ঘটনা কীভাবে পরের ঘটনাকে ডেকে আনল, তা লিখুন। তারপর প্রতিকার ও প্রতিরোধ লিখুন। Black Friday কেসে শৃঙ্খল: ট্রাফিক → HPA → Pending → নোড বিলম্ব → ডেটাবেস সংযোগ শেষ। GitOps image কেসে: স্বয়ংক্রিয় promotion + ভাঙা migration + alert-এর অভাব। etcd কেসে: একসাথে দুটি সদস্য ব্যর্থ — সাধারণ কারণ (যেমন একই ডিস্ক/zone) খুঁজুন। পূর্ণ নম্বর পেতে টাইমলাইনের প্রতিটি ধাপের সাথে আপনার সিদ্ধান্তের সম্পর্ক দেখান।

২.৪ Final Exam — মডেল উত্তর (সংক্ষিপ্ত)

Section A: প্রতিটি উত্তরের জন্য সংজ্ঞা + একটি উদাহরণ দিন। 99.9% হিসাব: 30 দিন = 43,200 মিনিট; 0.1% = 43.2 মিনিট = 43 মিনিট 12 সেকেন্ড।

Section B: প্রতিটি উত্তরে সিদ্ধান্ত, যুক্তি, বিকল্প ও ঝুঁকি — এই চারটি থাকতে হবে। শুধু সিদ্ধান্ত লিখলে অর্ধেক নম্বর।

Section C (নকশা): ভালো উত্তরের নকশায় থাকে — দুটি স্বাধীন ক্লাস্টার, কেন্দ্রীয় Git (GitOps), Argo CD/Flux, progressive delivery, কেন্দ্রীয় পর্যবেক্ষণ (OpenTelemetry), নীতি (OPA/Kyverno), সিক্রেট (ভল্ট/External

Secrets), FinOps ট্যাগিং ও বাজেট alert, এবং DR পরিকল্পনা (RTO/RPO-সহ)। বাদ দেওয়া বিকল্পের যুক্তি (যেমন কেন single cluster নয়, কেন mesh এখনই নয়) লিখলে পূর্ণ নম্বর।

২.৫ Evaluation Rubrics

স্তর	বৈশিষ্ট্য
চমৎকার (৮৫–১০০%)	সিদ্ধান্তের যুক্তি, বিকল্প ও ঝুঁকি — তিনটিই আছে; ব্যর্থতার পরিস্থিতি বিবেচনা করা হয়েছে।
ভালো (৭০–৮৪%)	সিদ্ধান্ত সঠিক, যুক্তি আছে, তবে বিকল্প বা ঝুঁকির আলোচনা আংশিক।
গ্রহণযোগ্য (৫০–৬৯%)	মূল ধারণা ঠিক, কিন্তু প্রয়োগ ও ট্রেড-অফের আলোচনা দুর্বল।
পুনরায় পড়ুন (৫০%-এর নিচে)	সংজ্ঞা মুখস্থ আছে, কিন্তু সিদ্ধান্ত নিতে পারছেন না — সংশ্লিষ্ট অধ্যায় আবার পড়ুন।

২.৬ Chapter Mapping

অধ্যায়	MCQ	Short Answer	Scenario / Case
অধ্যায় ১ — Kubernetes Control Plane	১–৫	১	—
অধ্যায় ২ — etcd ও API Server	৬–১০	২	Scenario ২; Case ৩
অধ্যায় ৩ — Multi-Cluster	১১–১৫	৩	Scenario ৭, ১৯; Case ৮
অধ্যায় ৪ — GitOps ও Argo CD	১৬–২০	৪, ২৫	Scenario ১, ১০; Case ২
অধ্যায় ৫ — Flux CD	২১–২৫	৫	Case ২
অধ্যায় ৬ — Helm ও Kustomize	২৬–৩০	৬	Scenario ৬
অধ্যায় ৭ — Service Mesh	৩১–৩৫	৭	Scenario ৫; Case ৪
অধ্যায় ৮ — Ingress, Gateway API ও Networking	৩৬–৪০	৮	Scenario ১৮
অধ্যায় ৯ — Storage (CSI)	৪১–৪৫	৯	Scenario ৯
অধ্যায় ১০ — Autoscaling ও Scheduling	৪৬–৫০	১০	Scenario ৮; Case ১
অধ্যায় ১১ — Progressive Delivery	৫১–৫৫	১১, ২৬	Scenario ৩
অধ্যায় ১২ — Backstage ও IDP	৫৬–৬০	১২, ২৯	Scenario ১৬; Case ১০
অধ্যায় ১৩ — Platform Engineering	৬১–৬৫	১৩, ২৯	Case ১০
অধ্যায় ১৪ — Enterprise CI/CD	৬৬–৭০	১৪, ৩০	—
অধ্যায় ১৫ — Terraform Enterprise	৭১–৭৫	১৫	Scenario ১৩
অধ্যায় ১৬ — Ansible	৭৬–৮০	১৬	—
অধ্যায় ১৭ — Observability	৮১–৮৫	১৭, ২৭	Scenario ১১; Case ৯

অধ্যায় ১৮ — SLO ও Error Budget	৮৬–৯০	১৮	Case ৯
অধ্যায় ১৯ — Security Governance	৯১–৯৫	১৯	Scenario ১২; Case ৮
অধ্যায় ২০ — FinOps	৯৬–১০০	২০	Scenario ৪; Case ৬
অধ্যায় ২১ — Disaster Recovery ও Chaos	১০১–১০৫	২১, ২৮	Scenario ৭, ১৭, ২০
অধ্যায় ২২ — Multi-Cloud ও Hybrid	১০৬–১১০	২২	—
অধ্যায় ২৩ — Zero-Trust ও Secrets	১১১–১১৫	২৩	Scenario ১৪; Case ৭
অধ্যায় ২৪ — Enterprise Transformation	১১৬–১২০	২৪	Case ১০